code2pdf-vscode

22 files, 3,097 lines

Printed 2026-10-11 from commit cb305d5 (main)

code2pdf-vscode/

CHANGELOG.md . . . 2
CLAUDE.md . . . 3
LICENSE . . . 5
README.md . . . 17
eslint.config.mjs . . . 20

images/src/

render.sh . . . 21

src/

args.ts . . . 22
cli.ts . . . 24
extension.ts . . . 28
git.ts . . . 41
install.ts . . . 44
progress.ts . . . 46
specs.ts . . . 49

src/test/integration/

commands.test.ts . . . 52

src/test/manual/

viewer.test.ts . . . 57

src/test/unit/

args.test.ts . . . 58
cli.test.ts . . . 60
git.test.ts . . . 63
helpers.ts . . . 65
install.test.ts . . . 66
progress.test.ts . . . 67
specs.test.ts . . . 70

CHANGELOG.md

Markdown, 29 lines, last changed 2026-10-11

```
# Changelog

## 2.0.0

A new extension, built on the [code2pdf](https://github.com/readbysearch/code2pdf) command-line
tool (3.12 or later) instead of Vim and Ghostscript.

- PDFs have a contents page, the files' own line numbers, their paths on every page, and a
  bookmark per file.
- Print a folder, files you check in a list, several selected files and folders, one file, or
  the lines you selected.
- Print what changed in git: uncommitted changes, the last commit, a branch, or everything since
  a commit. Changed lines are marked in the margin.
- If code2pdf isn't installed, or is too old, the extension says so and offers to install or
  update it in a terminal.
- A progress notification you can cancel, with the time so far always moving (even while
  code2pdf is quiet on a long file), warnings and errors shown instead of failing silently,
  and everything code2pdf printed in the **code2pdf** output panel.
- Settings: where PDFs go, what opens them, the colour theme, and globs to include or exclude.
  Version 1's settings still work if you set them.
- Works over Remote SSH and in WSL; on Windows itself it explains how to use WSL.

## 1.2.0

- Better syntax highlighting.

## 1.1.1

- First release: print a file or all source files with Vim and Ghostscript.
```

CLAUDE.md

Markdown, 62 lines, last changed 2026-10-11

```
# code2pdf for VS Code

A VS Code extension (Marketplace id `readbysearch.code2pdf`) that runs the code2pdf command-line
tool ([readbysearch/code2pdf](https://github.com/readbysearch/code2pdf)). It draws nothing
itself: every PDF comes from `code2pdf`, so features belong in the CLI first. User docs are in
README.md only.

## Layout

- `src/extension.ts`: commands, settings, messages; the only file that imports `vscode`.
- `src/cli.ts`: finds code2pdf (setting, PATH, the installers' folders), checks
  `MIN_VERSION`, runs it with progress and cancellation (SIGINT to its process group, then
  SIGKILL).
- `src/args.ts`: the command line for each kind of print; version 1's settings mapped to options.
- `src/git.ts`: the choices for Print Changes. A commit in the list means "it and everything
  after it" (`--changed <its parent>`); a typed REV means `--changed REV`.
- `src/progress.ts`: reads code2pdf's stderr (`[2/3] … (50%)`, `Done: N pages`, `Warning:`,
  `Note:`, `Error:`); `-s` reports on stdout.
- `src/install.ts`: install and update commands per platform and install kind.

## Rules

- Never fail silently (the old version's reviews were about that): every error reaches a
  message, with the command and code2pdf's output in the code2pdf output channel.
- Paths go to code2pdf relative to the workspace folder (cwd), so the PDF shows project paths.
- Options passed to code2pdf must exist in `code2pdf --help`: the unit test
  `every option used here is in code2pdf --help` checks it. Using a newer CLI feature means
  raising `MIN_VERSION` in `src/cli.ts` and saying so in the README.
- `--files-from` takes no `--include`/`--exclude`, and `--changed` takes no `--include`
  (code2pdf refuses them).
- Untrusted workspaces aren't supported: code2pdf runs git, and a repository's git config can
  run programs.

## Testing

```bash
npm ci
npm run lint
npm run test:unit            # node:test; the code2pdf-backed tests skip if it isn't installed
npx vscode-test              # integration, in a downloaded VS Code (.vscode-test/), on a git fixture
VSCODE_TEST_VERSION=1.90.0 npx vscode-test   # the oldest VS Code in engines
CODE2PDF_TEST_FILES='out/test/manual/*.test.js' npx vscode-test   # opens a PDF in the viewer
```

`.vscode-test.mjs` builds the fixture project (a git repo with two commits and an uncommitted
```

edit) and points `code2pdf.outputFolder` outside it. Integration tests stub `vscode.window` pickers and messages. CI (`.github/workflows/ci.yml`) runs on Ubuntu (the .deb; stable and 1.90.0) and macOS (Homebrew), and weekly against the latest code2pdf release.

Releasing

1. Bump `version` in package.json (above the version on the Marketplace) and add a CHANGELOG.md entry.
2. `npm run lint && npm run test:unit && npx vscode-test`, then `npx vsce package` and `npx vsce ls` to check the contents (only out/*.js, icon, README, CHANGELOG, LICENSE).
3. Commit as readbysearch and push `main` (`gh auth switch -u readbysearch`; switch back to waleking afterwards), wait for CI, then tag `vX.Y.Z` and push the tag.
4. Publish: upload the .vsix at https://marketplace.visualstudio.com/manage/publishers/readbysearch (signed in as the publisher's owner), or `npx vsce publish` with an Azure DevOps PAT (Marketplace: Manage scope, all accessible organizations; PATs expire).
5. Images in README.md are relative; vsce turns them into github.com/…/raw/HEAD/ links, so they must be pushed before the listing shows them. `images/src/render.sh` draws the icon.

LICENSE

Plain text, 661 lines, last changed 2026-10-11

```
                    GNU AFFERO GENERAL PUBLIC LICENSE
                       Version 3, 19 November 2007

 Copyright (C) 2007 Free Software Foundation, Inc. <https://fsf.org/>
 Everyone is permitted to copy and distribute verbatim copies
 of this license document, but changing it is not allowed.

                            Preamble

  The GNU Affero General Public License is a free, copyleft license for
software and other kinds of works, specifically designed to ensure
cooperation with the community in the case of network server software.

  The licenses for most software and other practical works are designed
to take away your freedom to share and change the works.  By contrast,
our General Public Licenses are intended to guarantee your freedom to
share and change all versions of a program--to make sure it remains free
software for all its users.

  When we speak of free software, we are referring to freedom, not
price.  Our General Public Licenses are designed to make sure that you
have the freedom to distribute copies of free software (and charge for
them if you wish), that you receive source code or can get it if you
want it, that you can change the software or use pieces of it in new
free programs, and that you know you can do these things.

  Developers that use our General Public Licenses protect your rights
with two steps: (1) assert copyright on the software, and (2) offer
you this License which gives you legal permission to copy, distribute
and/or modify the software.

  A secondary benefit of defending all users' freedom is that
improvements made in alternate versions of the program, if they
receive widespread use, become available for other developers to
incorporate.  Many developers of free software are heartened and
encouraged by the resulting cooperation.  However, in the case of
software used on network servers, this result may fail to come about.
The GNU General Public License permits making a modified version and
letting the public access it on a server without ever releasing its
source code to the public.

  The GNU Affero General Public License is designed specifically to
ensure that, in such cases, the modified source code becomes available
to the community.  It requires the operator of a network server to
provide the source code of the modified version running there to the
users of that server.  Therefore, public use of a modified version, on
a publicly accessible server, gives the public access to the source
code of the modified version.

  An older license, called the Affero General Public License and
published by Affero, was designed to accomplish similar goals.  This is
a different license, not a version of the Affero GPL, but Affero has
released a new version of the Affero GPL which permits relicensing under
this license.
```

```
55
56    The precise terms and conditions for copying, distribution and
57  modification follow.
58
59                         TERMS AND CONDITIONS
60
61    0. Definitions.
62
63    "This License" refers to version 3 of the GNU Affero General Public License.
64
65    "Copyright" also means copyright-like laws that apply to other kinds of
66  works, such as semiconductor masks.
67
68    "The Program" refers to any copyrightable work licensed under this
69  License.  Each licensee is addressed as "you".  "Licensees" and
70  "recipients" may be individuals or organizations.
71
72    To "modify" a work means to copy from or adapt all or part of the work
73  in a fashion requiring copyright permission, other than the making of an
74  exact copy.  The resulting work is called a "modified version" of the
75  earlier work or a work "based on" the earlier work.
76
77    A "covered work" means either the unmodified Program or a work based
78  on the Program.
79
80    To "propagate" a work means to do anything with it that, without
81  permission, would make you directly or secondarily liable for
82  infringement under applicable copyright law, except executing it on a
83  computer or modifying a private copy.  Propagation includes copying,
84  distribution (with or without modification), making available to the
85  public, and in some countries other activities as well.
86
87    To "convey" a work means any kind of propagation that enables other
88  parties to make or receive copies.  Mere interaction with a user through
89  a computer network, with no transfer of a copy, is not conveying.
90
91    An interactive user interface displays "Appropriate Legal Notices"
92  to the extent that it includes a convenient and prominently visible
93  feature that (1) displays an appropriate copyright notice, and (2)
94  tells the user that there is no warranty for the work (except to the
95  extent that warranties are provided), that licensees may convey the
96  work under this License, and how to view a copy of this License.  If
97  the interface presents a list of user commands or options, such as a
98  menu, a prominent item in the list meets this criterion.
99
100   1. Source Code.
101
102   The "source code" for a work means the preferred form of the work
103 for making modifications to it.  "Object code" means any non-source
104 form of a work.
105
106   A "Standard Interface" means an interface that either is an official
107 standard defined by a recognized standards body, or, in the case of
108 interfaces specified for a particular programming language, one that
109 is widely used among developers working in that language.
110
111   The "System Libraries" of an executable work include anything, other
112 than the work as a whole, that (a) is included in the normal form of
113 packaging a Major Component, but which is not part of that Major
```

Component, and (b) serves only to enable use of the work with that Major Component, or to implement a Standard Interface for which an implementation is available to the public in source code form. A "Major Component", in this context, means a major essential component (kernel, window system, and so on) of the specific operating system (if any) on which the executable work runs, or a compiler used to produce the work, or an object code interpreter used to run it.

The "Corresponding Source" for a work in object code form means all the source code needed to generate, install, and (for an executable work) run the object code and to modify the work, including scripts to control those activities. However, it does not include the work's System Libraries, or general-purpose tools or generally available free programs which are used unmodified in performing those activities but which are not part of the work. For example, Corresponding Source includes interface definition files associated with source files for the work, and the source code for shared libraries and dynamically linked subprograms that the work is specifically designed to require, such as by intimate data communication or control flow between those subprograms and other parts of the work.

The Corresponding Source need not include anything that users can regenerate automatically from other parts of the Corresponding Source.

The Corresponding Source for a work in source code form is that same work.

2. Basic Permissions.

All rights granted under this License are granted for the term of copyright on the Program, and are irrevocable provided the stated conditions are met. This License explicitly affirms your unlimited permission to run the unmodified Program. The output from running a covered work is covered by this License only if the output, given its content, constitutes a covered work. This License acknowledges your rights of fair use or other equivalent, as provided by copyright law.

You may make, run and propagate covered works that you do not convey, without conditions so long as your license otherwise remains in force. You may convey covered works to others for the sole purpose of having them make modifications exclusively for you, or provide you with facilities for running those works, provided that you comply with the terms of this License in conveying all material for which you do not control copyright. Those thus making or running the covered works for you must do so exclusively on your behalf, under your direction and control, on terms that prohibit them from making any copies of your copyrighted material outside their relationship with you.

Conveying under any other circumstances is permitted solely under the conditions stated below. Sublicensing is not allowed; section 10 makes it unnecessary.

3. Protecting Users' Legal Rights From Anti-Circumvention Law.

No covered work shall be deemed part of an effective technological measure under any applicable law fulfilling obligations under article 11 of the WIPO copyright treaty adopted on 20 December 1996, or similar laws prohibiting or restricting circumvention of such

measures.

  When you convey a covered work, you waive any legal power to forbid circumvention of technological measures to the extent such circumvention is effected by exercising rights under this License with respect to the covered work, and you disclaim any intention to limit operation or modification of the work as a means of enforcing, against the work's users, your or third parties' legal rights to forbid circumvention of technological measures.

  4. Conveying Verbatim Copies.

  You may convey verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice; keep intact all notices stating that this License and any non-permissive terms added in accord with section 7 apply to the code; keep intact all notices of the absence of any warranty; and give all recipients a copy of this License along with the Program.

  You may charge any price or no price for each copy that you convey, and you may offer support or warranty protection for a fee.

  5. Conveying Modified Source Versions.

  You may convey a work based on the Program, or the modifications to produce it from the Program, in the form of source code under the terms of section 4, provided that you also meet all of these conditions:

    a) The work must carry prominent notices stating that you modified it, and giving a relevant date.

    b) The work must carry prominent notices stating that it is released under this License and any conditions added under section 7. This requirement modifies the requirement in section 4 to "keep intact all notices".

    c) You must license the entire work, as a whole, under this License to anyone who comes into possession of a copy. This License will therefore apply, along with any applicable section 7 additional terms, to the whole of the work, and all its parts, regardless of how they are packaged. This License gives no permission to license the work in any other way, but it does not invalidate such permission if you have separately received it.

    d) If the work has interactive user interfaces, each must display Appropriate Legal Notices; however, if the Program has interactive interfaces that do not display Appropriate Legal Notices, your work need not make them do so.

  A compilation of a covered work with other separate and independent works, which are not by their nature extensions of the covered work, and which are not combined with it such as to form a larger program, in or on a volume of a storage or distribution medium, is called an "aggregate" if the compilation and its resulting copyright are not used to limit the access or legal rights of the compilation's users beyond what the individual works permit. Inclusion of a covered work in an aggregate does not cause this License to apply to the other parts of the aggregate.

```
  6. Conveying Non-Source Forms.

  You may convey a covered work in object code form under the terms
of sections 4 and 5, provided that you also convey the
machine-readable Corresponding Source under the terms of this License,
in one of these ways:

    a) Convey the object code in, or embodied in, a physical product
    (including a physical distribution medium), accompanied by the
    Corresponding Source fixed on a durable physical medium
    customarily used for software interchange.

    b) Convey the object code in, or embodied in, a physical product
    (including a physical distribution medium), accompanied by a
    written offer, valid for at least three years and valid for as
    long as you offer spare parts or customer support for that product
    model, to give anyone who possesses the object code either (1) a
    copy of the Corresponding Source for all the software in the
    product that is covered by this License, on a durable physical
    medium customarily used for software interchange, for a price no
    more than your reasonable cost of physically performing this
    conveying of source, or (2) access to copy the
    Corresponding Source from a network server at no charge.

    c) Convey individual copies of the object code with a copy of the
    written offer to provide the Corresponding Source.  This
    alternative is allowed only occasionally and noncommercially, and
    only if you received the object code with such an offer, in accord
    with subsection 6b.

    d) Convey the object code by offering access from a designated
    place (gratis or for a charge), and offer equivalent access to the
    Corresponding Source in the same way through the same place at no
    further charge.  You need not require recipients to copy the
    Corresponding Source along with the object code.  If the place to
    copy the object code is a network server, the Corresponding Source
    may be on a different server (operated by you or a third party)
    that supports equivalent copying facilities, provided you maintain
    clear directions next to the object code saying where to find the
    Corresponding Source.  Regardless of what server hosts the
    Corresponding Source, you remain obligated to ensure that it is
    available for as long as needed to satisfy these requirements.

    e) Convey the object code using peer-to-peer transmission, provided
    you inform other peers where the object code and Corresponding
    Source of the work are being offered to the general public at no
    charge under subsection 6d.

  A separable portion of the object code, whose source code is excluded
from the Corresponding Source as a System Library, need not be
included in conveying the object code work.

  A "User Product" is either (1) a "consumer product", which means any
tangible personal property which is normally used for personal, family,
or household purposes, or (2) anything designed or sold for incorporation
into a dwelling.  In determining whether a product is a consumer product,
doubtful cases shall be resolved in favor of coverage.  For a particular
product received by a particular user, "normally used" refers to a
```

typical or common use of that class of product, regardless of the status
of the particular user or of the way in which the particular user
actually uses, or expects or is expected to use, the product. A product
is a consumer product regardless of whether the product has substantial
commercial, industrial or non-consumer uses, unless such uses represent
the only significant mode of use of the product.

  "Installation Information" for a User Product means any methods,
procedures, authorization keys, or other information required to install
and execute modified versions of a covered work in that User Product from
a modified version of its Corresponding Source. The information must
suffice to ensure that the continued functioning of the modified object
code is in no case prevented or interfered with solely because
modification has been made.

  If you convey an object code work under this section in, or with, or
specifically for use in, a User Product, and the conveying occurs as
part of a transaction in which the right of possession and use of the
User Product is transferred to the recipient in perpetuity or for a
fixed term (regardless of how the transaction is characterized), the
Corresponding Source conveyed under this section must be accompanied
by the Installation Information. But this requirement does not apply
if neither you nor any third party retains the ability to install
modified object code on the User Product (for example, the work has
been installed in ROM).

  The requirement to provide Installation Information does not include a
requirement to continue to provide support service, warranty, or updates
for a work that has been modified or installed by the recipient, or for
the User Product in which it has been modified or installed. Access to a
network may be denied when the modification itself materially and
adversely affects the operation of the network or violates the rules and
protocols for communication across the network.

  Corresponding Source conveyed, and Installation Information provided,
in accord with this section must be in a format that is publicly
documented (and with an implementation available to the public in
source code form), and must require no special password or key for
unpacking, reading or copying.

  7. Additional Terms.

  "Additional permissions" are terms that supplement the terms of this
License by making exceptions from one or more of its conditions.
Additional permissions that are applicable to the entire Program shall
be treated as though they were included in this License, to the extent
that they are valid under applicable law. If additional permissions
apply only to part of the Program, that part may be used separately
under those permissions, but the entire Program remains governed by
this License without regard to the additional permissions.

  When you convey a copy of a covered work, you may at your option
remove any additional permissions from that copy, or from any part of
it. (Additional permissions may be written to require their own
removal in certain cases when you modify the work.) You may place
additional permissions on material, added by you to a covered work,
for which you have or can give appropriate copyright permission.

  Notwithstanding any other provision of this License, for material you

add to a covered work, you may (if authorized by the copyright holders of
that material) supplement the terms of this License with terms:

    a) Disclaiming warranty or limiting liability differently from the
    terms of sections 15 and 16 of this License; or

    b) Requiring preservation of specified reasonable legal notices or
    author attributions in that material or in the Appropriate Legal
    Notices displayed by works containing it; or

    c) Prohibiting misrepresentation of the origin of that material, or
    requiring that modified versions of such material be marked in
    reasonable ways as different from the original version; or

    d) Limiting the use for publicity purposes of names of licensors or
    authors of the material; or

    e) Declining to grant rights under trademark law for use of some
    trade names, trademarks, or service marks; or

    f) Requiring indemnification of licensors and authors of that
    material by anyone who conveys the material (or modified versions of
    it) with contractual assumptions of liability to the recipient, for
    any liability that these contractual assumptions directly impose on
    those licensors and authors.

  All other non-permissive additional terms are considered "further
restrictions" within the meaning of section 10. If the Program as you
received it, or any part of it, contains a notice stating that it is
governed by this License along with a term that is a further
restriction, you may remove that term. If a license document contains
a further restriction but permits relicensing or conveying under this
License, you may add to a covered work material governed by the terms
of that license document, provided that the further restriction does
not survive such relicensing or conveying.

  If you add terms to a covered work in accord with this section, you
must place, in the relevant source files, a statement of the
additional terms that apply to those files, or a notice indicating
where to find the applicable terms.

  Additional terms, permissive or non-permissive, may be stated in the
form of a separately written license, or stated as exceptions;
the above requirements apply either way.

  8. Termination.

  You may not propagate or modify a covered work except as expressly
provided under this License. Any attempt otherwise to propagate or
modify it is void, and will automatically terminate your rights under
this License (including any patent licenses granted under the third
paragraph of section 11).

  However, if you cease all violation of this License, then your
license from a particular copyright holder is reinstated (a)
provisionally, unless and until the copyright holder explicitly and
finally terminates your license, and (b) permanently, if the copyright
holder fails to notify you of the violation by some reasonable means
prior to 60 days after the cessation.

409
410 Moreover, your license from a particular copyright holder is
411 reinstated permanently if the copyright holder notifies you of the
412 violation by some reasonable means, this is the first time you have
413 received notice of violation of this License (for any work) from that
414 copyright holder, and you cure the violation prior to 30 days after
415 your receipt of the notice.
416
417 Termination of your rights under this section does not terminate the
418 licenses of parties who have received copies or rights from you under
419 this License. If your rights have been terminated and not permanently
420 reinstated, you do not qualify to receive new licenses for the same
421 material under section 10.
422
423 9. Acceptance Not Required for Having Copies.
424
425 You are not required to accept this License in order to receive or
426 run a copy of the Program. Ancillary propagation of a covered work
427 occurring solely as a consequence of using peer-to-peer transmission
428 to receive a copy likewise does not require acceptance. However,
429 nothing other than this License grants you permission to propagate or
430 modify any covered work. These actions infringe copyright if you do
431 not accept this License. Therefore, by modifying or propagating a
432 covered work, you indicate your acceptance of this License to do so.
433
434 10. Automatic Licensing of Downstream Recipients.
435
436 Each time you convey a covered work, the recipient automatically
437 receives a license from the original licensors, to run, modify and
438 propagate that work, subject to this License. You are not responsible
439 for enforcing compliance by third parties with this License.
440
441 An "entity transaction" is a transaction transferring control of an
442 organization, or substantially all assets of one, or subdividing an
443 organization, or merging organizations. If propagation of a covered
444 work results from an entity transaction, each party to that
445 transaction who receives a copy of the work also receives whatever
446 licenses to the work the party's predecessor in interest had or could
447 give under the previous paragraph, plus a right to possession of the
448 Corresponding Source of the work from the predecessor in interest, if
449 the predecessor has it or can get it with reasonable efforts.
450
451 You may not impose any further restrictions on the exercise of the
452 rights granted or affirmed under this License. For example, you may
453 not impose a license fee, royalty, or other charge for exercise of
454 rights granted under this License, and you may not initiate litigation
455 (including a cross-claim or counterclaim in a lawsuit) alleging that
456 any patent claim is infringed by making, using, selling, offering for
457 sale, or importing the Program or any portion of it.
458
459 11. Patents.
460
461 A "contributor" is a copyright holder who authorizes use under this
462 License of the Program or a work on which the Program is based. The
463 work thus licensed is called the contributor's "contributor version".
464
465 A contributor's "essential patent claims" are all patent claims
466 owned or controlled by the contributor, whether already acquired or
467 hereafter acquired, that would be infringed by some manner, permitted

by this License, of making, using, or selling its contributor version,
but do not include claims that would be infringed only as a
consequence of further modification of the contributor version. For
purposes of this definition, "control" includes the right to grant
patent sublicenses in a manner consistent with the requirements of
this License.

  Each contributor grants you a non-exclusive, worldwide, royalty-free
patent license under the contributor's essential patent claims, to
make, use, sell, offer for sale, import and otherwise run, modify and
propagate the contents of its contributor version.

  In the following three paragraphs, a "patent license" is any express
agreement or commitment, however denominated, not to enforce a patent
(such as an express permission to practice a patent or covenant not to
sue for patent infringement). To "grant" such a patent license to a
party means to make such an agreement or commitment not to enforce a
patent against the party.

  If you convey a covered work, knowingly relying on a patent license,
and the Corresponding Source of the work is not available for anyone
to copy, free of charge and under the terms of this License, through a
publicly available network server or other readily accessible means,
then you must either (1) cause the Corresponding Source to be so
available, or (2) arrange to deprive yourself of the benefit of the
patent license for this particular work, or (3) arrange, in a manner
consistent with the requirements of this License, to extend the patent
license to downstream recipients. "Knowingly relying" means you have
actual knowledge that, but for the patent license, your conveying the
covered work in a country, or your recipient's use of the covered work
in a country, would infringe one or more identifiable patents in that
country that you have reason to believe are valid.

  If, pursuant to or in connection with a single transaction or
arrangement, you convey, or propagate by procuring conveyance of, a
covered work, and grant a patent license to some of the parties
receiving the covered work authorizing them to use, propagate, modify
or convey a specific copy of the covered work, then the patent license
you grant is automatically extended to all recipients of the covered
work and works based on it.

  A patent license is "discriminatory" if it does not include within
the scope of its coverage, prohibits the exercise of, or is
conditioned on the non-exercise of one or more of the rights that are
specifically granted under this License. You may not convey a covered
work if you are a party to an arrangement with a third party that is
in the business of distributing software, under which you make payment
to the third party based on the extent of your activity of conveying
the work, and under which the third party grants, to any of the
parties who would receive the covered work from you, a discriminatory
patent license (a) in connection with copies of the covered work
conveyed by you (or copies made from those copies), or (b) primarily
for and in connection with specific products or compilations that
contain the covered work, unless you entered into that arrangement,
or that patent license was granted, prior to 28 March 2007.

  Nothing in this License shall be construed as excluding or limiting
any implied license or other defenses to infringement that may
otherwise be available to you under applicable patent law.

12. No Surrender of Others' Freedom.

If conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot convey a covered work so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not convey it at all. For example, if you agree to terms that obligate you to collect a royalty for further conveying from those to whom you convey the Program, the only way you could satisfy both those terms and this License would be to refrain entirely from conveying the Program.

13. Remote Network Interaction; Use with the GNU General Public License.

Notwithstanding any other provision of this License, if you modify the Program, your modified version must prominently offer all users interacting with it remotely through a computer network (if your version supports such interaction) an opportunity to receive the Corresponding Source of your version by providing access to the Corresponding Source from a network server at no charge, through some standard or customary means of facilitating copying of software. This Corresponding Source shall include the Corresponding Source for any work covered by version 3 of the GNU General Public License that is incorporated pursuant to the following paragraph.

Notwithstanding any other provision of this License, you have permission to link or combine any covered work with a work licensed under version 3 of the GNU General Public License into a single combined work, and to convey the resulting work. The terms of this License will continue to apply to the part which is the covered work, but the work with which it is combined will remain governed by version 3 of the GNU General Public License.

14. Revised Versions of this License.

The Free Software Foundation may publish revised and/or new versions of the GNU Affero General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies that a certain numbered version of the GNU Affero General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that numbered version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of the GNU Affero General Public License, you may choose any version ever published by the Free Software Foundation.

If the Program specifies that a proxy can decide which future versions of the GNU Affero General Public License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Program.

Later license versions may give you additional or different permissions. However, no additional obligations are imposed on any author or copyright holder as a result of your choosing to follow a later version.

586
587 15. Disclaimer of Warranty.
588
589 THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY
590 APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT
591 HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY
592 OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO,
593 THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
594 PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM
595 IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF
596 ALL NECESSARY SERVICING, REPAIR OR CORRECTION.
597
598 16. Limitation of Liability.
599
600 IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING
601 WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MODIFIES AND/OR CONVEYS
602 THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY
603 GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE
604 USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF
605 DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD
606 PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS),
607 EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF
608 SUCH DAMAGES.
609
610 17. Interpretation of Sections 15 and 16.
611
612 If the disclaimer of warranty and limitation of liability provided
613 above cannot be given local legal effect according to their terms,
614 reviewing courts shall apply local law that most closely approximates
615 an absolute waiver of all civil liability in connection with the
616 Program, unless a warranty or assumption of liability accompanies a
617 copy of the Program in return for a fee.
618
619 END OF TERMS AND CONDITIONS
620
621 How to Apply These Terms to Your New Programs
622
623 If you develop a new program, and you want it to be of the greatest
624 possible use to the public, the best way to achieve this is to make it
625 free software which everyone can redistribute and change under these terms.
626
627 To do so, attach the following notices to the program. It is safest
628 to attach them to the start of each source file to most effectively
629 state the exclusion of warranty; and each file should have at least
630 the "copyright" line and a pointer to where the full notice is found.
631
632 <one line to give the program's name and a brief idea of what it does.>
633 Copyright (C) <year> <name of author>
634
635 This program is free software: you can redistribute it and/or modify
636 it under the terms of the GNU Affero General Public License as published by
637 the Free Software Foundation, either version 3 of the License, or
638 (at your option) any later version.
639
640 This program is distributed in the hope that it will be useful,
641 but WITHOUT ANY WARRANTY; without even the implied warranty of
642 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
643 GNU Affero General Public License for more details.
644

```
    You should have received a copy of the GNU Affero General Public License
    along with this program.  If not, see <https://www.gnu.org/licenses/>.

Also add information on how to contact you by electronic and paper mail.

  If your software can interact with users remotely through a computer
network, you should also make sure that it provides a way for users to
get its source.  For example, if your program is a web application, its
interface could display a "Source" link that leads users to an archive
of the code.  There are many ways you could offer source, and different
solutions will be better for different programs; see section 13 for the
specific requirements.

  You should also get your employer (if you work as a programmer) or school,
if any, to sign a "copyright disclaimer" for the program, if necessary.
For more information on this, and how to apply and follow the GNU AGPL, see
<https://www.gnu.org/licenses/>.
```

README.md

Markdown, 94 lines, last changed 2026-10-11

```
# code2pdf for VS Code

Print source code as a PDF made for reading on paper or a tablet: a contents page, syntax
highlighting, each file's own line numbers, its path on every page, and a bookmark per file.
Print a folder, a few files, the lines you selected, or what changed in git, with the changed
lines marked in the margin.



The extension runs the [code2pdf](https://github.com/readbysearch/code2pdf) command-line tool,
on macOS and Linux. Your code stays on your machine.

## Install code2pdf first

The extension needs code2pdf 3.12 or later. If it isn't installed, the extension offers to
install it in a terminal, where you can see the command:

| System | Command |
| --- | --- |
| macOS, or Linux with Homebrew | `brew install readbysearch/tap/code2pdf` |
| Ubuntu 24.04+, Debian 13+ | `curl -fL -o /tmp/code2pdf_all.deb https://github.com/readbysearch/code2pdf/releases/latest/download/code2pdf_all.deb && sudo apt install /tmp/code2pdf_all.deb` |
| Other Linux, no sudo | `curl -sSL https://raw.githubusercontent.com/readbysearch/code2pdf/main/install.sh \| bash -s -- --user` |

On Windows, use [WSL](https://code.visualstudio.com/docs/remote/wsl): open your folder in WSL,
install code2pdf there, and print from that window. Over Remote SSH, code2pdf runs on the remote
machine.

## Printing

Right-click in the Explorer, an editor, an editor tab or the Source Control menu, or run the
commands from the Command Palette (they start with "code2pdf").

- **A folder**: right-click it and choose **Print to PDF**. code2pdf picks the source files: it
  leaves out hidden folders, build output, vendored dependencies, binary files and data files such
  as JSON. The `include` and `exclude` settings change that.
- **Chosen files**: right-click a folder and choose **Choose Files and Print to PDF…**. Every file
  code2pdf would print is listed and checked; uncheck the ones to leave out.
- **Several files or folders**: select them in the Explorer, then **Print to PDF**. They print as
  one PDF, sorted by path, each folder with code2pdf's usual selection.
- **A file**: right-click it, its tab or its editor, and choose **Print File to PDF**.
- **Some lines**: select them and choose **Print Selection to PDF**. They print with their real
```

```
  line numbers, and several selections print together.
- **Changes**: choose **Print Changes to PDF…** from the Source Control menu or the Command
  Palette, then what to print:
  - your uncommitted changes;
  - the last commit;
  - this branch since it left `main`;
  - a recent commit and everything after it;
  - or the changes since a commit, tag or branch you name.

  Each change prints with the code around it, such as the whole function it's in, and the
  changed lines are marked with a bar in the margin. Files that git doesn't track yet are
   left
  out, with a note.

A progress notification shows the page count as code2pdf works. Cancel it to stop. When it's
done, the PDF opens in your PDF viewer, and any warnings from code2pdf are shown. Everything
code2pdf printed is in the **code2pdf** output panel.

## Settings

| Setting | Default | What it does |
| --- | --- | --- |
| `code2pdf.outputFolder` | empty | A folder to save PDFs in without asking. Empty: you
   choose where each PDF goes. |
| `code2pdf.openWith` | `viewer` | Open the PDF in your PDF `viewer`, in a VS Code tab
   (`editor`, needs a PDF viewer extension), or `none`. |
| `code2pdf.theme` | `paper` | `paper`, `mono` (black and white, for printers), or any
   Pygments style, such as `github-dark`. |
| `code2pdf.include` | `[]` | Globs to add when printing a folder: `*.json`, `.github`,
   `vendor/mylib`. |
| `code2pdf.exclude` | `[]` | Globs to leave out when printing a folder or changes:
   `tests`, `*_test.go`, `docs/**/*.md`. |
| `code2pdf.executable` | empty | The path to `code2pdf`, if it isn't on your `PATH` or in
   the usual install folders. |

Without `/`, a glob matches names at any depth; with `/`, it matches the path from the
   folder.
Put `include` and `exclude` in a workspace's `.vscode/settings.json` to set them per
   project.

### Coming from version 1

Version 1 drew PDFs with Vim and Ghostscript. Version 2 uses code2pdf instead, which you
install once (see above). The old settings (`blacklisted_folders`,
   `whitelisted_file_extensions`
and so on) still work if you set them. You no longer need them: code2pdf's default selection
covers common source code, and `include` and `exclude` cover the rest.

## Also from code2pdf

- On the command line, `code2pdf --help` lists everything, including `--summary` for a line
  under the title, and `code2txt` for one Markdown file to give an LLM.
- The [Claude Code skill](https://github.com/readbysearch/code2pdf#using-code2pdf-from-a-
   coding-agent) lets Claude
  print code for you: "print my last commit for review, with the changes marked".

## Problems and ideas

```

```
89 [Open an issue](https://github.com/readbysearch/code2pdf-vscode/issues). If a print fails,
   the
90 **code2pdf** output panel has the command that ran and everything it printed.
91
92 ## License
93
94 AGPL-3.0-only, like code2pdf.
```

eslint.config.mjs

JavaScript, 15 lines, last changed 2026-10-11

```
import js from "@eslint/js";
import tseslint from "typescript-eslint";

export default tseslint.config(
  { ignores: ["out/", ".vscode-test/", "node_modules/"] },
  js.configs.recommended,
  ...tseslint.configs.recommended,
  {
    rules: {
      eqeqeq: "error",
      curly: "error",
      "@typescript-eslint/no-unused-vars": ["error", { argsIgnorePattern: "^_" }],
    },
  },
);
```

images/src/

render.sh

Bash, 15 lines, last changed 2026-10-11

```bash
#!/bin/bash
# Renders the icon (and README images) from the files here. Needs WeasyPrint and poppler.
# Usage: images/src/render.sh  (from the repository root)
set -euo pipefail
here="$(cd "$(dirname "$0")" && pwd)"
python="${PYTHON:-python3}"
tmp="$(mktemp -d)"; trap 'rm -rf "$tmp"' EXIT
"$python" - "$here/icon.svg" "$tmp/icon.pdf" <<'PY'
import sys, weasyprint
svg, out = sys.argv[1:]
html = f'<style>@page {{ size: 256px 256px; margin: 0 }} body {{ margin: 0 }}</style><img
   src="file://{svg}" style="width:256px;height:256px;display:block">'
weasyprint.HTML(string=html).write_pdf(out)
PY
pdftocairo -png -transp -singlefile -scale-to 256 "$tmp/icon.pdf" "$here/../icon"
echo "wrote images/icon.png"
```

src/

args.ts

TypeScript, 94 lines, last changed 2026-10-11

```ts
// code2pdf command lines for each kind of print. Pure, so the unit tests can check them.

import { argPath } from "./specs";

/** The settings that change code2pdf's command line. */
export interface PrintSettings {
  theme: string;
  include: readonly string[];
  exclude: readonly string[];
  /** From the settings of version 1, if the user set any. */
  legacy: LegacySettings;
}

/** Version 1's settings. Only values the user set are passed; their old defaults are now code2pdf's. */
export interface LegacySettings {
  blacklistedFolders?: readonly string[];
  blacklistedFolderPattern?: string;
  whitelistedFileExtensions?: readonly string[];
  whitelistedFileNames?: readonly string[];
}

/** code2pdf options with the same effect as version 1's settings. */
export function legacyArgs(legacy: LegacySettings): string[] {
  const args: string[] = [];
  const folders = clean(legacy.blacklistedFolders);
  if (folders.length) {
    args.push("--ignore-folders", folders.join(","));
  }
  const pattern = legacy.blacklistedFolderPattern?.trim();
  if (pattern && pattern !== "env*") {  // env* folders are skipped by default
    args.push("--exclude", pattern);
  }
  const extensions = clean(legacy.whitelistedFileExtensions).map((e) => e.replace(/^\*?\./, ""));
  if (extensions.length) {
    args.push("--include-types", extensions.join(","));
  }
  for (const name of clean(legacy.whitelistedFileNames)) {
    args.push("--include", name);
  }
  return args;
}

function clean(list: readonly string[] | undefined): string[] {
  return (list ?? []).map((s) => s.trim()).filter(Boolean);
}

function themeArgs(theme: string): string[] {
  const name = theme.trim();
  return name && name !== "paper" ? ["--theme", name] : [];
}

```

```ts
function excludeArgs(exclude: readonly string[]): string[] {
  return clean(exclude).flatMap((glob) => ["--exclude", glob]);
}

/** Options that choose a folder's files. */
export function selectionArgs(settings: PrintSettings): string[] {
  return [
    ...clean(settings.include).flatMap((glob) => ["--include", glob]),
    ...excludeArgs(settings.exclude),
    ...legacyArgs(settings.legacy),
  ];
}

/** One file, or part of one (`app.py:10-40`). */
export function fileArgs(spec: string, output: string, settings: PrintSettings): string[] {
  return ["-s", argPath(spec), "-o", output, ...themeArgs(settings.theme)];
}

/** A folder, with code2pdf's default selection and the include and exclude settings. */
export function folderArgs(folder: string, output: string, settings: PrintSettings):
    string[] {
  return ["-a", argPath(folder), "-o",
    output, ...themeArgs(settings.theme), ...selectionArgs(settings)];
}

/** The files `folderArgs` would print, without printing them. */
export function listArgs(folder: string, settings: PrintSettings): string[] {
  return ["-a", argPath(folder), "--list-files", ...selectionArgs(settings)];
}

/** Exactly the files given on stdin, relative to FOLDER, in that order. */
export function filesFromArgs(folder: string, output: string, settings: PrintSettings):
    string[] {
  return ["-a", argPath(folder), "--files-from", "-", "-o",
    output, ...themeArgs(settings.theme)];
}

/** What changed in FOLDER since REV, with the changed lines marked. */
export function changedArgs(rev: string, folder: string, output: string, settings:
    PrintSettings): string[] {
  return ["--changed", rev, "-a", argPath(folder), "-o",
    output, ...themeArgs(settings.theme),
     ...excludeArgs(settings.exclude)];
}

/** A command line for the output log, quoted the way a shell would need it. */
export function shellQuote(args: readonly string[]): string {
  return args.map((a) => (/^[\w@%+=:,./~-]+$/.test(a) ? a : `'${a.replace(/'/g, `'\
    \'`)}'`)).join(" ");
}
```

src/

cli.ts

TypeScript, 212 lines, last changed 2026-10-11

```ts
// Finding the code2pdf command, checking its version, and running it. No vscode imports.

import { execFile, spawn } from "node:child_process";
import * as os from "node:os";
import * as path from "node:path";
import { OutputReader, ProgressUpdate } from "./progress";
import { expandHome } from "./specs";

/** The oldest code2pdf with everything this extension uses (`--changed` came in 3.12). */
export const MIN_VERSION: readonly [number, number] = [3, 12];

export interface Code2pdf {
  /** The command to run: a path, or "code2pdf" when it was found on PATH. */
  command: string;
  version: string;
  /** "Homebrew", "Debian package", "install.sh", "git checkout", … as `code2pdf --version` says. */
  installKind: string;
  installDir: string;
}

/** No code2pdf where we looked. */
export class NotInstalled extends Error {
  constructor(readonly looked: readonly string[], readonly fromSetting: boolean) {
    super(fromSetting ? `code2pdf isn't at ${looked[0]}` : "code2pdf isn't installed");
  }
}

/** A code2pdf older than MIN_VERSION. */
export class TooOld extends Error {
  constructor(readonly found: Code2pdf) {
    super(`code2pdf ${found.version} is older than ${MIN_VERSION.join(".")}`);
  }
}

/** Version and install details from `code2pdf --version` output. */
export function parseVersion(output: string): Omit<Code2pdf, "command"> | undefined {
  const first = output.match(/^code2pdf (\S+)/m);
  if (!first) {
    return undefined;
  }
  const version = first[1].startsWith("(") ? "unknown" : first[1];
  const installed = output.match(/^\s*installed: ([^,\n]+)(?:, (.+))?$/m);
  return {
    version,
    installKind: installed?.[1].trim() ?? "unknown",
    installDir: installed?.[2]?.trim() ?? "",
  };
}

/**
 * Whether VERSION ("3.12", "3.12.1", "3.12+3") is MIN or later. Versions that aren't numbers
```

```ts
 * ("unreleased", "unknown") come from git checkouts, which are taken to be new enough.
 */
export function isNewEnough(version: string, min: readonly [number, number] = MIN_VERSION): boolean {
  const m = version.match(/^(\d+)\.(\d+)/);
  if (!m) {
    return !/^\d/.test(version);
  }
  const major = Number(m[1]);
  const minor = Number(m[2]);
  return major > min[0] || (major === min[0] && minor >= min[1]);
}

/**
 * Where to look for code2pdf: the setting if there is one; otherwise PATH, then the folders the
 * installers use, for when VS Code was started without the login shell's PATH.
 */
export function candidates(setting: string, home = os.homedir(), platform = process.platform): string[] {
  if (setting.trim()) {
    return [expandHome(setting.trim(), home)];
  }
  if (platform === "win32") {
    return ["code2pdf"];
  }
  return [
    "code2pdf",
    "/opt/homebrew/bin/code2pdf",
    "/usr/local/bin/code2pdf",
    "/home/linuxbrew/.linuxbrew/bin/code2pdf",
    path.join(home, ".local", "bin", "code2pdf"),
    "/usr/bin/code2pdf",
  ];
}

/** `COMMAND --version` output, or undefined if COMMAND can't be run. */
function versionOutput(command: string): Promise<string | undefined> {
  return new Promise((resolve) => {
    execFile(command, ["--version"], { timeout: 60_000 }, (error, stdout, stderr) => {
      const code = (error as NodeJS.ErrnoException | null)?.code;
      if (code === "ENOENT" || code === "EACCES" || code === "ENOTDIR") {
        resolve(undefined);
      } else {
        resolve(`${stdout}\n${stderr}`);
      }
    });
  });
}

/** Finds code2pdf, or throws NotInstalled or TooOld. */
export async function find(setting: string): Promise<Code2pdf> {
  const looked = candidates(setting);
  for (const command of looked) {
    const output = await versionOutput(command);
    if (output === undefined) {
      continue;
    }
    const parsed = parseVersion(output);
```

```
    // Before 3.7 there was no --version, so a code2pdf that runs but doesn't say is old
    const found: Code2pdf = { command, ...(parsed ?? { version: "older than 3.7",
     installKind: "unknown", installDir: "" }) };
    if (!parsed || !isNewEnough(parsed.version)) {
      throw new TooOld(found);
    }
    return found;
  }
  throw new NotInstalled(looked, Boolean(setting.trim()));
}

/** Like vscode.CancellationToken, without depending on vscode. */
export interface Cancellation {
  isCancellationRequested: boolean;
  onCancellationRequested(listener: () => void): unknown;
}

export interface RunOptions {
  cwd: string;
  args: readonly string[];
  /** Written to code2pdf's stdin (the list for `--files-from -`). */
  input?: string;
  onProgress?: (update: ProgressUpdate) => void;
  /** Everything code2pdf writes, for the output log. */
  onOutput?: (text: string) => void;
  cancellation?: Cancellation;
  /** Stdout is data (a file list), not messages. */
  quietStdout?: boolean;
  /** How long to wait after Ctrl-C before killing it. */
  graceMs?: number;
}

export interface RunResult {
  code: number | null;
  cancelled: boolean;
  stdout: string;
  reader: OutputReader;
}

/**
 * Runs code2pdf. Cancelling sends Ctrl-C (SIGINT) to its whole process group, as a terminal
 * would, so code2pdf stops cleanly and removes its temporary files; it is killed if it
   hasn't
 * stopped after a few seconds.
 */
export function run(command: string, options: RunOptions): Promise<RunResult> {
  return new Promise((resolve, reject) => {
    const reader = new OutputReader(options.onProgress);
    const ownGroup = process.platform !== "win32";
    const child = spawn(command, [...options.args], {
      cwd: options.cwd,
      detached: ownGroup,
      stdio: ["pipe", "pipe", "pipe"],
    });
    let stdout = "";
    let cancelled = false;
    let killTimer: NodeJS.Timeout | undefined;

    const signal = (name: NodeJS.Signals) => {
```

```
      try {
        if (ownGroup && child.pid) {
          process.kill(-child.pid, name);
        } else {
          child.kill(name);
        }
      } catch {
        // It has already exited
      }
    };
    options.cancellation?.onCancellationRequested(() => {
      if (cancelled || child.exitCode !== null) {
        return;
      }
      cancelled = true;
      signal("SIGINT");
      killTimer = setTimeout(() => signal("SIGKILL"), options.graceMs ?? 5000);
    });

    child.stdout.setEncoding("utf8");
    child.stderr.setEncoding("utf8");
    child.stdout.on("data", (text: string) => {
      stdout += text;
      if (!options.quietStdout) {
        reader.feed(text, "stdout");
      }
      options.onOutput?.(text);
    });
    child.stderr.on("data", (text: string) => {
      reader.feed(text);
      options.onOutput?.(text);
    });
    child.stdin.on("error", () => {
      // code2pdf exited before reading all of stdin; its exit code says why
    });
    child.on("error", (error) => {
      clearTimeout(killTimer);
      reject(error);
    });
    child.on("close", (code) => {
      clearTimeout(killTimer);
      reader.end();
      resolve({ code, cancelled, stdout, reader });
    });

    child.stdin.end(options.input ?? "");
  });
}
```

src/

extension.ts

TypeScript, 641 lines, last changed 2026-10-11

```ts
import * as fs from "node:fs";
import * as os from "node:os";
import * as path from "node:path";
import * as vscode from "vscode";
import * as args from "./args";
import * as cli from "./cli";
import * as git from "./git";
import { INSTALL_GUIDE, installCommand, parseOsRelease, updateCommand } from "./install";
import { type OutputReader, withElapsed } from "./progress";
import * as specs from "./specs";

let log: vscode.OutputChannel;
let state: vscode.Memento;
let found: cli.Code2pdf | undefined;

/** A print, ready to run. */
interface Job {
  /** Shown while it runs, such as "Printing src/app.py". */
  title: string;
  cwd: string;
  args: string[];
  /** For `--files-from -`. */
  input?: string;
  output: string;
}

/** Thrown to stop a command with a message for the user (none when they cancelled). */
class Stop extends Error {
  constructor(message: string, readonly level: "info" | "warning" | "error" = "info") {
    super(message);
  }
}

export function activate(context: vscode.ExtensionContext) {
  log = vscode.window.createOutputChannel("code2pdf");
  state = context.globalState;
  // Each returns the PDF's path, or undefined if nothing was printed
  const register = (id: string, handler: (...a: never[]) => Promise<string | undefined>) =>
    context.subscriptions.push(vscode.commands.registerCommand(id, (...a: unknown[]) =>
   guard(() => handler(...(a as never[])))));
  register("code2pdf.print", printResources);
  register("code2pdf.chooseFiles", chooseFiles);
  register("code2pdf.printFile", printFile);
  register("code2pdf.printSelection", printSelection);
  register("code2pdf.printWorkspace", printWorkspace);
  register("code2pdf.printChanges", printChanges);
  // Version 1's commands, for keybindings that use them
  register("extension.printFileToPdf", () => printFile());
  register("extension.printAll", () => printWorkspace());
  context.subscriptions.push(
    log,
    vscode.workspace.onDidChangeConfiguration((e) => {
      if (e.affectsConfiguration("code2pdf.executable")) {
```

```
        found = undefined;
      }
    }),
  );
}

export function deactivate() {}

/** Runs a command and reports every failure, so nothing fails silently. Returns the PDF's path. */
async function guard(command: () => Promise<string | undefined>): Promise<string | undefined> {
  try {
    return await command();
  } catch (error) {
    if (error instanceof Stop) {
      if (error.message) {
        void showWithOutput(error.level, error.message, false);
      }
      return undefined;
    }
    const message = error instanceof Error ? error.message : String(error);
    log.appendLine(`Error: ${error instanceof Error && error.stack ? error.stack : message}`);
    void showWithOutput("error", `code2pdf: ${message}`);
    return undefined;
  }
}

/** Shows a message without waiting for it to be dismissed. */
function show(level: "error" | "warning" | "info", message: string, ...buttons: string[]): Thenable<string | undefined> {
  if (level === "error") {
    return vscode.window.showErrorMessage(message, ...buttons);
  }
  return level === "warning" ? vscode.window.showWarningMessage(message, ...buttons) : vscode.window.showInformationMessage(message, ...buttons);
}

async function showWithOutput(level: "error" | "warning" | "info", message: string, withButton = true): Promise<void> {
  if ((await show(level, message, ...(withButton ? ["Show Output"] : []))) === "Show Output") {
    log.show(true);
  }
}

// Settings

function settings(scope?: vscode.Uri) {
  const config = vscode.workspace.getConfiguration("code2pdf", scope);
  const set = <T>(key: string): T | undefined => {
    const seen = config.inspect<T>(key);
    return seen?.workspaceFolderValue ?? seen?.workspaceValue ?? seen?.globalValue;
  };
  const print: args.PrintSettings = {
    theme: config.get<string>("theme", "paper"),
    include: config.get<string[]>("include", []),
    exclude: config.get<string[]>("exclude", []),
```

```
    legacy: {
      blacklistedFolders: set<string[]>("blacklisted_folders"),
      blacklistedFolderPattern: set<string>("blacklisted_folder_pattern"),
      whitelistedFileExtensions: set<string[]>("whitelisted_file_extensions"),
      whitelistedFileNames: set<string[]>("whitelisted_file_names"),
    },
  };
  return {
    print,
    outputFolder: config.get<string>("outputFolder", "").trim(),
    openWith: config.get<"viewer" | "editor" | "none">("openWith", "viewer"),
    executable: config.get<string>("executable", ""),
  };
}

// Finding code2pdf

async function code2pdf(): Promise<cli.Code2pdf | undefined> {
  if (found) {
    return found;
  }
  if (process.platform === "win32") {
    const choice = await vscode.window.showWarningMessage(
      "code2pdf runs on macOS and Linux. On Windows, open your folder in WSL (the WSL extension), install code2pdf there, and print from that window.",
      "About WSL");
    if (choice) {
      void vscode.env.openExternal(vscode.Uri.parse("https://code.visualstudio.com/docs/remote/wsl"));
    }
    return undefined;
  }
  const executable = settings().executable;
  try {
    found = await cli.find(executable);
    log.appendLine(`Using ${found.command}: code2pdf ${found.version} (${found.installKind})`);
    return found;
  } catch (error) {
    if (error instanceof cli.NotInstalled) {
      await offerInstall(error);
      return undefined;
    }
    if (error instanceof cli.TooOld) {
      await offerUpdate(error.found);
      return undefined;
    }
    throw error;
  }
}

async function hasBrew(): Promise<boolean> {
  for (const brew of ["/opt/homebrew/bin/brew", "/usr/local/bin/brew", "/home/linuxbrew/.linuxbrew/bin/brew"]) {
    if (fs.existsSync(brew)) {
      return true;
    }
  }
```

```
  return (process.env.PATH ?? "").split(path.delimiter).some((dir) => dir &&
    fs.existsSync(path.join(dir, "brew")));
}

async function offerInstall(error: cli.NotInstalled): Promise<void> {
  log.appendLine(`code2pdf not found; looked for: ${error.looked.join(", ")}`);
  if (error.fromSetting) {
    const choice = await vscode.window.showErrorMessage(
      `code2pdf isn't at ${error.looked[0]}, where the code2pdf.executable setting says it
    is.`, "Open Settings");
    if (choice) {
      void vscode.commands.executeCommand("workbench.action.openSettings",
    "code2pdf.executable");
    }
    return;
  }
  let release;
  try {
    release = parseOsRelease(fs.readFileSync("/etc/os-release", "utf8"));
  } catch {
    release = undefined;
  }
  const install = installCommand(process.platform, await hasBrew(), release);
  const message = "This extension prints with the code2pdf command-line tool, which isn't
    installed." +
    (install ? ` It can be installed with: ${install.command}` : process.platform ===
    "darwin" ? " It installs with Homebrew (brew.sh)." : "");
  const buttons = [...(install ? [install.label] : []), "Installation Guide"];
  const choice = await vscode.window.showWarningMessage(message, ...buttons);
  if (install && choice === install.label) {
    runInTerminal("Install code2pdf", install.command);
  } else if (choice === "Installation Guide") {
    void vscode.env.openExternal(vscode.Uri.parse(INSTALL_GUIDE));
  }
}

async function offerUpdate(old: cli.Code2pdf): Promise<void> {
  const update = updateCommand(old, os.homedir());
  const message = `This extension needs code2pdf ${cli.MIN_VERSION.join(".")} or later, and
    ${old.command} is ${old.version}.` +
    (update ? ` It can be updated with: ${update.command}` : "");
  const buttons = [...(update ? [update.label] : []), "Installation Guide"];
  const choice = await vscode.window.showWarningMessage(message, ...buttons);
  if (update && choice === update.label) {
    runInTerminal("Update code2pdf", update.command);
  } else if (choice === "Installation Guide") {
    void vscode.env.openExternal(vscode.Uri.parse(INSTALL_GUIDE));
  }
}

function runInTerminal(name: string, command: string): void {
  const terminal = vscode.window.createTerminal({ name });
  terminal.show();
  terminal.sendText(command);
  found = undefined;
  void vscode.window.showInformationMessage(`${name} is running in the terminal. When it's
    done, print again.`);
}

```

```
211 // Where things are
212
213 function workspaceFolderOf(uri: vscode.Uri): string | undefined {
214   return vscode.workspace.getWorkspaceFolder(uri)?.uri.fsPath;
215 }
216
217 function isFolder(p: string): boolean {
218   try {
219     return fs.statSync(p).isDirectory();
220   } catch {
221     return false;
222   }
223 }
224
225 function onDisk(uri: vscode.Uri | undefined): vscode.Uri {
226   if (!uri) {
227     throw new Stop("Open a file to print first.");
228   }
229   if (uri.scheme === "untitled") {
230     throw new Stop("Save the file first: code2pdf prints files on disk.");
231   }
232   if (uri.scheme !== "file") {
233     throw new Stop(`code2pdf prints files on disk, and this one is a ${uri.scheme} file.`);
234   }
235   return uri;
236 }
237
238 async function pickWorkspaceFolder(placeHolder: string): Promise<vscode.Uri> {
239   const folders = vscode.workspace.workspaceFolders ?? [];
240   if (!folders.length) {
241     throw new Stop("Open a folder first.");
242   }
243   if (folders.length === 1) {
244     return folders[0].uri;
245   }
246   const folder = await vscode.window.showWorkspaceFolderPick({ placeHolder });
247   if (!folder) {
248     throw new Stop("");
249   }
250   return folder.uri;
251 }
252
253 /** Where to save NAME: the outputFolder setting, or a save dialog. Undefined if cancelled.
      */
254 async function outputPath(name: string, workspace: string | undefined): Promise<string |
      undefined> {
255   const home = os.homedir();
256   const folder = settings().outputFolder;
257   if (folder) {
258     const absolute = specs.expandHome(folder, home);
259     return path.join(path.isAbsolute(absolute) ? absolute : path.join(workspace ?? home,
      absolute), name);
260   }
261   const last = state.get<string>("lastFolder");
262   const start = last && isFolder(last) ? last : [path.join(home, "Desktop"),
      path.join(home, "Documents")].find(isFolder) ?? home;
263   const uri = await vscode.window.showSaveDialog({
264     defaultUri: vscode.Uri.file(path.join(start, name)),
265     filters: { PDF: ["pdf"] },
```

```
    saveLabel: "Print",
    title: "Save the PDF",
  });
  if (!uri) {
    return undefined;
  }
  await state.update("lastFolder", path.dirname(uri.fsPath));
  return uri.fsPath.toLowerCase().endsWith(".pdf") ? uri.fsPath : `${uri.fsPath}.pdf`;
}

/** Saves DOCUMENT if the user wants to; throws Stop if they cancel. */
async function savedOrStop(uri: vscode.Uri, linesMustMatch: boolean): Promise<void> {
  const document = vscode.workspace.textDocuments.find((d) => d.uri.toString() ===
   uri.toString());
  if (!document?.isDirty) {
    return;
  }
  const name = path.basename(uri.fsPath);
  const buttons = linesMustMatch ? ["Save and Print"] : ["Save and Print", "Print Saved
   Version"];
  const choice = await vscode.window.showWarningMessage(
    linesMustMatch ? `${name} has unsaved changes. Save it first, so the line numbers
   match.` : `${name} has unsaved changes.`,
    { modal: true }, ...buttons);
  if (choice === "Save and Print") {
    if (!(await document.save())) {
      throw new Stop(`${name} couldn't be saved.`, "error");
    }
  } else if (choice !== "Print Saved Version") {
    throw new Stop("");
  }
}

// Commands

/** Explorer: the clicked item, or all selected items. */
async function printResources(clicked?: vscode.Uri, selected?: vscode.Uri[]):
   Promise<string | undefined> {
  const uris = selected?.length ? selected : clicked ? [clicked] : [];
  if (!uris.length) {
    return printFile();
  }
  if (uris.length === 1) {
    const uri = onDisk(uris[0]);
    return isFolder(uri.fsPath) ? printFolder(uri) : printFile(uri);
  }
  return printMany(uris.map(onDisk));
}

async function printFile(uri?: vscode.Uri): Promise<string | undefined> {
  const file = onDisk(uri ?? vscode.window.activeTextEditor?.document.uri);
  if (isFolder(file.fsPath)) {
    return printFolder(file);
  }
  if (!(await code2pdf())) {
    return undefined;
  }
  await savedOrStop(file, false);
  const root = workspaceFolderOf(file) ?? path.dirname(file.fsPath);
```

```
  const rel = specs.relativePath(root, file.fsPath);
  const output = await outputPath(`${specs.fileNamePart(specs.stem(file.fsPath))}.pdf`,
   root);
  if (!output) {
    return undefined;
  }
  return execute({ title: `Printing ${rel}`, cwd: root, args: args.fileArgs(rel, output,
   settings(file).print), output });
}

async function printSelection(): Promise<string | undefined> {
  const editor = vscode.window.activeTextEditor;
  if (!editor) {
    throw new Stop("Open a file and select the lines to print first.");
  }
  const file = onDisk(editor.document.uri);
  const ranges = specs.selectionRanges(editor.selections.map((s) => ({
    startLine: s.start.line, endLine: s.end.line, endCharacter: s.end.character, isEmpty:
   s.isEmpty,
  })));
  if (!ranges.length) {
    throw new Stop("Select the lines to print first.");
  }
  if (!(await code2pdf())) {
    return undefined;
  }
  await savedOrStop(file, true);
  const root = workspaceFolderOf(file) ?? path.dirname(file.fsPath);
  const rel = specs.relativePath(root, file.fsPath);
  const spec = specs.rangeSpec(rel, ranges);
  const output = await outputPath(specs.selectionPdfName(file.fsPath, ranges), root);
  if (!output) {
    return undefined;
  }
  return execute({ title: `Printing ${spec}`, cwd: root, args: args.fileArgs(spec, output,
   settings(file).print), output });
}

async function printWorkspace(): Promise<string | undefined> {
  return printFolder(await pickWorkspaceFolder("Print which folder?"));
}

async function printFolder(folder: vscode.Uri): Promise<string | undefined> {
  if (!(await code2pdf())) {
    return undefined;
  }
  const root = workspaceFolderOf(folder) ?? folder.fsPath;
  const rel = specs.relativePath(root, folder.fsPath);
  const output = await outputPath(`$
   {specs.fileNamePart(path.basename(folder.fsPath))}.pdf`, root);
  if (!output) {
    return undefined;
  }
  return execute({ title: `Printing ${rel === "." ? path.basename(root) : rel}`, cwd: root,
   args: args.folderArgs(rel, output, settings(folder).print), output });
}

/** The files code2pdf would print from FOLDER (relative to it), with the include and
   exclude settings. */
```

```
373 async function listFiles(command: string, cwd: string, folder: string, scope: vscode.Uri):
      Promise<string[]> {
374   const listArgs = args.listArgs(folder, settings(scope).print);
375   log.appendLine(`\n$ ${args.shellQuote([command, ...listArgs])}    (in ${cwd})`);
376   const result = await cli.run(command, { cwd, args: listArgs, quietStdout: true, onOutput:
       undefined });
377   for (const line of [...result.reader.warnings.map((w) => `Warning: ${w}
      `), ...result.reader.errors.map((e) => `Error: ${e}`)]) {
378     log.appendLine(line);
379   }
380   if (result.code !== 0) {
381     throw new Error(result.reader.errorMessage() ?? `code2pdf --list-files exited with code
      ${result.code}`);
382   }
383   return result.stdout.split("\n").map((l) => l.trim()).filter(Boolean);
384 }
385
386 /** Several selected files and folders: each folder's usual files, then printed as one list.
      */
387 async function printMany(uris: vscode.Uri[]): Promise<string | undefined> {
388   const command = await code2pdf();
389   if (!command) {
390     return undefined;
391   }
392   const paths = uris.map((u) => u.fsPath);
393   const folders = new Set(uris.map((u) => workspaceFolderOf(u)));
394   const root = folders.size === 1 && [...folders][0] ? ([...folders][0] as string) :
       specs.commonFolder(paths, isFolder);
395   const items = paths.map((p) => ({ path: p, rel: specs.relativePath(root, p) })).sort((a,
       b) => specs.treeOrder(a.rel, b.rel));
396   const list: string[] = [];
397   for (const item of items) {
398     if (isFolder(item.path)) {
399       const inside = await listFiles(command.command, root, item.rel,
      vscode.Uri.file(item.path));
400       list.push(...inside.map((f) => (item.rel === "." ? f : `${item.rel}/${f}`)));
401     } else {
402       list.push(item.rel);
403     }
404   }
405   const files = specs.unique(list);
406   if (!files.length) {
407     throw new Stop("There's nothing to print in the selected folders: code2pdf's selection
      found no source files.");
408   }
409   const output = await outputPath(`$
       {specs.fileNamePart(path.basename(root))}-selected.pdf`, root);
410   if (!output) {
411     return undefined;
412   }
413   return execute({
414     title: `Printing ${files.length} file${files.length === 1 ? "" : "s"}`,
415     cwd: root,
416     args: args.filesFromArgs(".", output, settings(uris[0]).print),
417     input: files.join("\n") + "\n",
418     output,
419   });
420 }
421
```

```
/** A folder's files in a list to uncheck, then the checked ones printed. */
async function chooseFiles(uri?: vscode.Uri): Promise<string | undefined> {
  const folder = uri ? onDisk(uri) : await pickWorkspaceFolder("Choose files from which folder?");
  if (!isFolder(folder.fsPath)) {
    return printFile(folder);
  }
  const command = await code2pdf();
  if (!command) {
    return undefined;
  }
  const root = workspaceFolderOf(folder) ?? folder.fsPath;
  const rel = specs.relativePath(root, folder.fsPath);
  const files = await vscode.window.withProgress(
    { location: vscode.ProgressLocation.Window, title: "code2pdf: listing files" },
    () => listFiles(command.command, root, rel, folder));
  if (!files.length) {
    throw new Stop(`code2pdf's selection found no source files in ${rel === "." ? path.basename(root) : rel}.`);
  }
  const picked = await vscode.window.showQuickPick(
    files.map((f) => ({ label: f, picked: true })),
    { canPickMany: true, title: `Print from ${rel === "." ? path.basename(root) : rel}`, placeHolder: "Uncheck the files to leave out, then press Enter", matchOnDescription: true });
  if (!picked?.length) {
    return undefined;
  }
  const output = await outputPath(`${specs.fileNamePart(path.basename(folder.fsPath))}.pdf`, root);
  if (!output) {
    return undefined;
  }
  return execute({
    title: `Printing ${picked.length} file${picked.length === 1 ? "" : "s"}`,
    cwd: root,
    args: args.filesFromArgs(rel, output, settings(folder).print),
    input: picked.map((p) => p.label).join("\n") + "\n",
    output,
  });
}

/** What printChanges accepts: a Source Control repository (from its menu), a folder, or options. */
type ChangesTarget = { rootUri?: vscode.Uri } | vscode.Uri | { folder?: string; rev?: string } | undefined;

async function printChanges(target?: ChangesTarget): Promise<string | undefined> {
  let folder: string;
  let rev: string | undefined;
  if (target instanceof vscode.Uri) {
    folder = onDisk(target).fsPath;
  } else if (target && "rootUri" in target && target.rootUri) {
    folder = target.rootUri.fsPath;
  } else if (target && "folder" in target && target.folder) {
    folder = target.folder;
    rev = "rev" in target ? target.rev : undefined;
  } else {
    folder = (await pickWorkspaceFolder("Print changes in which folder?")).fsPath;
```

```
  }
  if (!(await git.repositoryRoot(folder))) {
    throw new Stop(`${path.basename(folder)} isn't in a git repository, so there are no changes to print.`, "warning");
  }
  if (!(await code2pdf())) {
    return undefined;
  }
  let name: string;
  if (rev) {
    name = `${path.basename(folder)}-since-${rev}`;
  } else {
    const picked = await pickChanges(folder);
    if (!picked) {
      return undefined;
    }
    ({ rev, name } = picked);
  }
  const output = await outputPath(`${specs.fileNamePart(name)}.pdf`, folder);
  if (!output) {
    return undefined;
  }
  return execute({ title: "Printing changes", cwd: folder, args: args.changedArgs(rev, ".", output, settings(vscode.Uri.file(folder)).print), output });
}

async function pickChanges(folder: string): Promise<{ rev: string; name: string } | undefined> {
  const choices = await git.changeChoices(folder);
  if (!choices.main.length && !choices.commits.length) {
    throw new Stop("This repository has no commits yet, so there are no changes to compare with.");
  }
  type Item = vscode.QuickPickItem & { choice?: git.ChangeChoice; enter?: boolean };
  const items: Item[] = [
    ...choices.main.map((c) => ({ label: c.label, description: c.description, detail: c.detail, choice: c })),
    { label: "From a commit to now", kind: vscode.QuickPickItemKind.Separator },
    ...choices.commits.map((c) => ({ label: c.label, description: c.description, choice: c })),
    { label: "", kind: vscode.QuickPickItemKind.Separator },
    { label: "$(edit) Since a commit, tag or branch you name…", enter: true },
  ];
  const picked = await vscode.window.showQuickPick(items, {
    title: "Print which changes?",
    placeHolder: "Changed lines are marked in the margin. A commit below means it and everything after it.",
    matchOnDescription: true,
  });
  if (!picked) {
    return undefined;
  }
  if (picked.choice) {
    return { rev: picked.choice.rev, name: picked.choice.name };
  }
  const rev = await vscode.window.showInputBox({
    title: "Print the changes made since…",
    prompt: "A commit, tag or branch. Its own changes are left out; what came after it is printed.",
```

```
    placeHolder: "v1.2.0, main, HEAD~3, a1b2c3d",
    validateInput: async (value) => (!value.trim() || (await git.isCommit(folder, value.trim()))) ? undefined : `git doesn't know ${value.trim()}`,
  });
  if (!rev?.trim()) {
    return undefined;
  }
  return { rev: rev.trim(), name: `${path.basename(folder)}-since-${rev.trim()}` };
}

// Running a print

async function execute(job: Job): Promise<string | undefined> {
  const command = await code2pdf();
  if (!command) {
    return undefined;
  }
  log.appendLine(`\n$ ${args.shellQuote([command.command, ...job.args])}${job.input ? " < list" : ""}    (in ${job.cwd})`);
  if (job.input) {
    const lines = job.input.trimEnd().split("\n");
    log.appendLine(lines.slice(0, 50).map((l) => `    ${l}`).join("\n") + (lines.length > 50 ? `\n    … ${lines.length - 50} more` : ""));
  }
  const result = await vscode.window.withProgress(
    { location: vscode.ProgressLocation.Notification, title: job.title, cancellable: true },
    async (progress, token) => {
      // code2pdf can go quiet for a minute (laying out a long file, writing the PDF), so the
      // elapsed time ticks every second and the notification never looks frozen
      const started = Date.now();
      let done = 0;
      let message: string | undefined;
      const report = (increment = 0) =>
        progress.report({ message: withElapsed(message, (Date.now() - started) / 1000), increment });
      const ticker = setInterval(report, 1000);
      try {
        return await cli.run(command.command, {
          cwd: job.cwd,
          args: job.args,
          input: job.input,
          cancellation: token,
          onOutput: (text) => log.append(text),
          onProgress: (update) => {
            const increment = update.fraction === undefined ? 0 : Math.max(0, update.fraction * 100 - done);
            done += increment;
            message = update.message ?? message;
            report(increment);
          },
        });
      } catch (error) {
        found = undefined;  // it was uninstalled or moved: look for it again next time
        const code = (error as NodeJS.ErrnoException).code;
        throw code === "ENOENT" ? new Stop(`code2pdf is no longer at ${command.command}. Print again to look for it.`, "error") : error;
      } finally {
        clearInterval(ticker);
```

```
      }
    });
  if (result.cancelled) {
    log.appendLine("Stopped.");
    vscode.window.setStatusBarMessage("code2pdf: stopped; nothing was written", 5000);
    return undefined;
  }
  if (result.code !== 0) {
    if (result.code === 127 || result.code === 126) {
      found = undefined;
    }
    void showWithOutput("error", `code2pdf: ${result.reader.errorMessage() ?? `stopped with exit code ${result.code}`}`);
    return undefined;
  }
  if (!fs.existsSync(job.output)) {
    void showWithOutput("error", `code2pdf finished, but there's no PDF at ${job.output}.`);
    return undefined;
  }
  await finished(job.output, result.reader);
  return job.output;
}

/** Opens the PDF as the openWith setting says, and says what code2pdf warned about. */
async function finished(pdf: string, reader: OutputReader): Promise<void> {
  const name = path.basename(pdf);
  const pages = reader.pages ? `, ${reader.pages.toLocaleString("en-US")} page${reader.pages === 1 ? "" : "s"}` : "";
  const remarks = [...reader.warnings, ...reader.notes];
  const remark = remarks.length ? ` ${remarks[0]}${remarks.length > 1 ? ` (and ${remarks.length - 1} more in the output)` : ""}` : "";
  const level = reader.warnings.length ? "warning" : "info";
  const openWith = settings().openWith;
  const remote = vscode.env.remoteName;
  const uri = vscode.Uri.file(pdf);

  if (openWith === "viewer" && !remote) {
    await vscode.env.openExternal(uri);
  } else if (openWith === "editor") {
    await vscode.commands.executeCommand("vscode.open", uri);
  }
  const opened = (openWith === "viewer" && !remote) || openWith === "editor";
  if (opened && !remarks.length) {
    vscode.window.setStatusBarMessage(`$(check) code2pdf: ${name}${pages}`, 8000);
    return;
  }
  const where = remote ? `${pdf} (on ${remote})` : pdf;
  const message = opened ? `Saved ${name}${pages}.${remark}` : `Saved ${where}${pages}.${remark}`;
  const buttons = [
    ...(opened ? [] : remote ? ["Open in VS Code"] : ["Open"]),
    ...(remote ? ["Copy Path"] : [process.platform === "darwin" ? "Reveal in Finder" : "Show in Folder"]),
    ...(remarks.length ? ["Show Output"] : []),
  ];
  // Not awaited: the command is done, whether or not the message is ever clicked
  void show(level, message, ...buttons).then((choice) => {
    if (choice === "Open") {
      void vscode.env.openExternal(uri);
```

```
    } else if (choice === "Open in VS Code") {
      void vscode.commands.executeCommand("vscode.open", uri);
    } else if (choice === "Copy Path") {
      void vscode.env.clipboard.writeText(pdf);
    } else if (choice === "Reveal in Finder" || choice === "Show in Folder") {
      void vscode.commands.executeCommand("revealFileInOS", uri);
    } else if (choice === "Show Output") {
      log.show(true);
    }
  });
}
```

src/

git.ts

TypeScript, 157 lines, last changed 2026-10-11

```ts
// The choices for "Print Changes to PDF…", read from git. No vscode imports.

import { execFile } from "node:child_process";
import * as path from "node:path";
import { fileNamePart } from "./specs";

export class GitError extends Error {}

/** Runs git in CWD and returns its output, without the final newline. */
export function git(cwd: string, args: readonly string[]): Promise<string> {
  return new Promise((resolve, reject) => {
    execFile("git", [...args], { cwd, maxBuffer: 16 * 1024 * 1024 }, (error, stdout, stderr) => {
      if (error) {
        const code = (error as NodeJS.ErrnoException).code;
        reject(new GitError(code === "ENOENT" ? "git isn't installed" : stderr.trim() || error.message));
      } else {
        resolve(stdout.replace(/\n$/, ""));
      }
    });
  });
}

/** git's answer, or undefined if git says no. */
async function maybe(cwd: string, args: readonly string[]): Promise<string | undefined> {
  try {
    return await git(cwd, args);
  } catch (error) {
    if (error instanceof GitError && error.message === "git isn't installed") {
      throw error;
    }
    return undefined;
  }
}

/** The top folder of the repository FOLDER is in, or undefined if it isn't in one. */
export function repositoryRoot(folder: string): Promise<string | undefined> {
  return maybe(folder, ["rev-parse", "--show-toplevel"]);
}

/** Whether REV names a commit (a hash, tag, branch, HEAD~2, …). */
export async function isCommit(folder: string, rev: string): Promise<boolean> {
  return !rev.startsWith("-") && (await maybe(folder, ["rev-parse", "--verify", "--quiet", `${rev}^{commit}`])) !== undefined;
}

export interface ChangeChoice {
  label: string;
  description?: string;
  detail?: string;
  /** For `code2pdf --changed`: changes made since this commit are printed. */
  rev: string;
```

```ts
  /** For the PDF's file name. */
  name: string;
}

export interface Commit {
  hash: string;
  short: string;
  subject: string;
  when: string;
  /** The first parent's hash, or undefined for a root commit. */
  parent?: string;
}

/** Recent commits, newest first. */
export async function recentCommits(folder: string, count = 30): Promise<Commit[]> {
  const out = await maybe(folder, ["log", `-n${count}`, "--format=%H%x1f%h%x1f%s%x1f%cr%x1f%P"]);
  if (!out) {
    return [];
  }
  return out.split("\n").map((line) => {
    const [hash, short, subject, when, parents] = line.split("\x1f");
    return { hash, short, subject, when, parent: parents.split(" ")[0] || undefined };
  });
}

/** The branch this one was made from: origin's default branch, or a local main or master. */
async function baseBranch(folder: string): Promise<string | undefined> {
  const remoteHead = await maybe(folder, ["symbolic-ref", "--quiet", "--short", "refs/remotes/origin/HEAD"]);
  if (remoteHead) {
    return remoteHead;
  }
  for (const name of ["main", "master"]) {
    if (await maybe(folder, ["rev-parse", "--verify", "--quiet", `refs/heads/${name}`])) {
      return name;
    }
  }
  return undefined;
}

export interface Choices {
  /** The fixed choices: uncommitted changes, the last commit, this branch. */
  main: ChangeChoice[];
  /** Recent commits: picking one prints it and everything after it. */
  commits: ChangeChoice[];
}

/** What FOLDER's repository offers to print. Empty when it has no commits yet. */
export async function changeChoices(folder: string): Promise<Choices> {
  const repo = fileNamePart(path.basename((await repositoryRoot(folder)) ?? folder));
  const commits = await recentCommits(folder);
  if (!commits.length) {
    return { main: [], commits: [] };
  }
  const head = commits[0];
  const main: ChangeChoice[] = [];

```

```
  // Changes to tracked files only: code2pdf leaves untracked files out, with a note
  const status = (await maybe(folder, ["status", "--porcelain", "--", "."])) ?? "";
  const lines = status.split("\n").filter(Boolean);
  const untracked = lines.filter((l) => l.startsWith("??")).length;
  const changed = lines.length - untracked;
  if (changed) {
    const stat = (await maybe(folder, ["diff", "HEAD", "--shortstat", "--", "."]))?.trim();
    main.push({
      label: "Uncommitted changes",
      description: stat,
      detail: untracked ? `New files that aren't added to git yet are left out (${untracked}); git add them to include them` : undefined,
      rev: "HEAD",
      name: `${repo}-uncommitted`,
    });
  }

  if (head.parent) {
    main.push({
      label: "The last commit",
      description: `${head.short} ${head.subject}`,
      detail: changed ? "With your uncommitted changes too: code2pdf compares with the files as they are now" : undefined,
      rev: head.parent,
      name: `${repo}-${head.short}`,
    });
  }

  const branch = await maybe(folder, ["rev-parse", "--abbrev-ref", "HEAD"]);
  const base = await baseBranch(folder);
  if (branch && branch !== "HEAD" && base && base !== branch && base.replace(/^origin\//, "") !== branch) {
    const fork = await maybe(folder, ["merge-base", "HEAD", base]);
    const ahead = fork && fork !== head.hash ? Number(await maybe(folder, ["rev-list", "--count", `${fork}..HEAD`])) : 0;
    if (fork && ahead > 0) {
      main.push({
        label: `This branch, ${branch}`,
        description: `${ahead} commit${ahead === 1 ? "" : "s"} since it left ${base}`,
        rev: fork,
        name: `${repo}-${fileNamePart(branch)}`,
      });
    }
  }

  const fromCommits = commits
    .filter((c) => c.parent)
    .map((c) => ({
      label: c.subject,
      description: `${c.short}, ${c.when}`,
      rev: c.parent as string,
      name: `${repo}-from-${c.short}`,
    }));
  return { main, commits: fromCommits };
}
```

src/

install.ts

TypeScript, 80 lines, last changed 2026-10-11

```ts
// Which command installs or updates code2pdf on this machine. Pure, so the unit tests can check
// each platform.

import type { Code2pdf } from "./cli";

export const INSTALL_GUIDE = "https://github.com/readbysearch/code2pdf#installation";

const DEB_URL = "https://github.com/readbysearch/code2pdf/releases/latest/download/code2pdf_all.deb";
const INSTALL_SH = "https://raw.githubusercontent.com/readbysearch/code2pdf/main/install.sh";

export interface InstallCommand {
  /** Button label, such as "Install with Homebrew". */
  label: string;
  /** Run in a terminal, where the user sees it and can answer prompts (sudo, for one). */
  command: string;
}

/** What `/etc/os-release` says, as far as it matters here. */
export interface OsRelease {
  id: string;
  versionId: string;
}

export function parseOsRelease(text: string): OsRelease {
  const field = (name: string) => text.match(new RegExp(`^${name}="?([^"\\n]*)"?$`, "m"))?.[1] ?? "";
  return { id: field("ID"), versionId: field("VERSION_ID") };
}

/** Whether the .deb installs here: it needs Ubuntu 24.04 or later, or Debian 13 or later. */
export function debWorks(release: OsRelease): boolean {
  const version = parseFloat(release.versionId);
  return (release.id === "ubuntu" && version >= 24.04) || (release.id === "debian" && version >= 13);
}

const brew = (verb: "install" | "upgrade"): InstallCommand => ({
  label: verb === "install" ? "Install with Homebrew" : "Update with Homebrew",
  command: `brew ${verb} readbysearch/tap/code2pdf`,
});

const deb = (verb: "Install" | "Update"): InstallCommand => ({
  label: `${verb} the .deb`,
  command: `curl -fL -o /tmp/code2pdf_all.deb ${DEB_URL} && sudo apt install /tmp/code2pdf_all.deb`,
});

const installScript = (verb: "Install" | "Update", where: "--user" | "--system"): InstallCommand => ({
```

```
  label: where === "--user" ? `${verb} (no sudo)` : verb,
  command: `curl -sSL ${INSTALL_SH} | bash -s -- ${where}`,
});

/** How to install code2pdf here, or undefined when only the guide can help. */
export function installCommand(platform: NodeJS.Platform, hasBrew: boolean, release?:
   OsRelease): InstallCommand | undefined {
  if (platform === "darwin") {
    return hasBrew ? brew("install") : undefined;
  }
  if (platform !== "linux") {
    return undefined;
  }
  if (hasBrew) {
    return brew("install");
  }
  if (release && debWorks(release)) {
    return deb("Install");
  }
  return installScript("Install", "--user");
}

/** How to update the code2pdf that was FOUND, the same way it was installed. */
export function updateCommand(found: Code2pdf, home: string): InstallCommand | undefined {
  const kind = found.installKind;
  if (kind === "Homebrew") {
    return brew("upgrade");
  }
  if (kind === "Debian package") {
    return deb("Update");
  }
  if (kind.startsWith("install.sh")) {
    return installScript("Update", found.installDir.startsWith(home + "/") ? "--user" : "--
   system");
  }
  return undefined;
}
```

src/

progress.ts

TypeScript, 136 lines, last changed 2026-10-11

```ts
// Reads what code2pdf writes while it works. It reports on stderr, one line per step:
//
//   code2pdf: 11 files, 2,918 lines (~62 pages)
//     [1/3] Preparing layout…  0:00 elapsed
//     [2/3] Laying out page 7 of ~62 (10%), 0:02 elapsed
//     [3/3] Writing PDF…  0:05 elapsed, ~0:02 left
//   Done: 61 pages in 0:07 → /path/to/out.pdf
//
// plus "Warning: …", "Note: …" and "Error: …" lines. `-s` prints "Converting to a PDF file for X"
// and "PDF created at X" on stdout instead.

export interface ProgressUpdate {
  /** Shown under the progress title. */
  message?: string;
  /** How much is done, from 0 to 1. */
  fraction?: number;
}

/** Minutes and seconds: 75 -> "1:15". */
export function clock(seconds: number): string {
  const whole = Math.floor(seconds);
  return `${Math.floor(whole / 60)}:${String(whole % 60).padStart(2, "0")}`;
}

/** The progress message with the time so far: "Laying out page 7 of ~62 (0:21)". */
export function withElapsed(message: string | undefined, seconds: number): string {
  return message ? `${message} (${clock(seconds)})` : `${clock(seconds)} elapsed`;
}

const ELAPSED = /[,\s]+\d+:\d\d elapsed.*$/;

/** What one line of code2pdf output says about progress, if anything. */
export function parseProgress(line: string): ProgressUpdate | undefined {
  const text = line.trim();
  let m = text.match(/^code2pdf: ([\d,]+ files?), [\d,]+ lines? \((~[\d,]+ pages?)\)/);
  if (m) {
    return { message: `${m[1]}, ${m[2]}`, fraction: 0 };
  }
  m = text.match(/^\[(\d)\/3\] (.*)$/);
  if (m) {
    const stage = Number(m[1]);
    const step = m[2].replace(ELAPSED, "").replace(/…$/, "").trim();
    const percent = step.match(/\((\d+)%\)/);
    const message = step.replace(/\s*\(\d+%\)/, "");
    if (stage === 1) {
      return { message, fraction: 0.03 };
    }
    if (stage === 2) {
      return { message, fraction: 0.05 + 0.85 * (percent ? Number(percent[1]) / 100 : 0) };
    }
    return { message, fraction: 0.92 };
  }
```

```
  m = text.match(/^in (\d+) parts of/);
  if (m) {
    return { message: `In ${m[1]} parts` };
  }
  if (/^Done: /.test(text) || /^PDF created at /.test(text)) {
    return { fraction: 1 };
  }
  m = text.match(/^Converting to a PDF file for (.*)$/);
  if (m) {
    return { message: m[1] };
  }
  return undefined;
}

/** Collects code2pdf's output as it arrives: progress, warnings, notes, errors and page count. */
export class OutputReader {
  readonly warnings: string[] = [];
  readonly notes: string[] = [];
  readonly errors: string[] = [];
  pages: number | undefined;
  /** Lines that weren't progress, for an error message when there's no "Error:" line. */
  private readonly other: string[] = [];
  private readonly partial = { stdout: "", stderr: "" };

  constructor(private readonly onProgress?: (update: ProgressUpdate) => void) {}

  /** Feed a chunk of output. Lines may be split across chunks. */
  feed(chunk: string, stream: "stdout" | "stderr" = "stderr"): void {
    const lines = (this.partial[stream] + chunk).split(/\r\n|\r|\n/);
    this.partial[stream] = lines.pop() ?? "";
    for (const line of lines) {
      this.line(line);
    }
  }

  /** Call once the process has exited, for last lines without a newline. */
  end(): void {
    for (const stream of ["stdout", "stderr"] as const) {
      if (this.partial[stream]) {
        this.line(this.partial[stream]);
        this.partial[stream] = "";
      }
    }
  }

  private line(line: string): void {
    const text = line.trim();
    if (!text) {
      return;
    }
    const update = parseProgress(line);
    if (update) {
      this.onProgress?.(update);
    }
    const done = text.match(/^Done: ([\d,]+) pages? in /);
    if (done) {
      this.pages = Number(done[1].replace(/,/g, ""));
      return;
```

```
    }
    if (update) {
      return;
    }
    if (text.startsWith("Warning: ")) {
      addOnce(this.warnings, text.slice("Warning: ".length));
    } else if (text.startsWith("Note: ")) {
      addOnce(this.notes, text.slice("Note: ".length));
    } else if (text.startsWith("Error: ")) {
      addOnce(this.errors, text.slice("Error: ".length));
    } else {
      this.other.push(text);
    }
  }

  /** The best one-line explanation of a failure, if code2pdf gave one. */
  errorMessage(): string | undefined {
    return this.errors[0] ?? this.other[this.other.length - 1];
  }
}

function addOnce(list: string[], text: string): void {
  if (!list.includes(text)) {
    list.push(text);
  }
}
```

src/

specs.ts

TypeScript, 124 lines, last changed 2026-10-11

```ts
// Pure helpers for what to print and what to call it. Nothing here imports vscode, so the unit
// tests run in plain Node.

import * as path from "node:path";

/** Lines START to END, counted from 1, both included. */
export interface LineRange {
  start: number;
  end: number;
}

/** The parts of a VS Code selection that matter here (lines counted from 0, as VS Code does). */
export interface SelectionLike {
  startLine: number;
  endLine: number;
  endCharacter: number;
  isEmpty: boolean;
}

/**
 * The lines to print for an editor's selections: counted from 1, sorted, overlaps merged.
 * A selection that ends at the very start of a line (as when whole lines are selected by
 * dragging down) doesn't include that line.
 */
export function selectionRanges(selections: readonly SelectionLike[]): LineRange[] {
  const ranges = selections
    .filter((s) => !s.isEmpty)
    .map((s) => {
      const last = s.endCharacter === 0 && s.endLine > s.startLine ? s.endLine - 1 : s.endLine;
      return { start: s.startLine + 1, end: last + 1 };
    })
    .sort((a, b) => a.start - b.start);
  const merged: LineRange[] = [];
  for (const range of ranges) {
    const last = merged[merged.length - 1];
    if (last && range.start <= last.end + 1) {
      last.end = Math.max(last.end, range.end);
    } else {
      merged.push({ ...range });
    }
  }
  return merged;
}

/** code2pdf's way to name part of a file: `src/app.py:10-40,90-120`. */
export function rangeSpec(file: string, ranges: readonly LineRange[]): string {
  const parts = ranges.map((r) => (r.start === r.end ? `${r.start}` : `${r.start}-${r.end}`));
  return `${file}:${parts.join(",")}`;
}
```

```ts

/** A path as code2pdf should see it: forward slashes, and never taken for an option. */
export function argPath(p: string): string {
  const slashed = p.split(path.sep).join("/");
  return slashed.startsWith("-") ? `./${slashed}` : slashed;
}

/** FILE relative to ROOT with forward slashes ("." for ROOT itself). */
export function relativePath(root: string, file: string): string {
  const rel = path.relative(root, file);
  return rel === "" ? "." : rel.split(path.sep).join("/");
}

/** Whether FILE is ROOT or inside it. */
export function isInside(root: string, file: string): boolean {
  const rel = path.relative(root, file);
  return rel === "" || (!rel.startsWith("..") && !path.isAbsolute(rel));
}

/** The deepest folder that contains all of PATHS (each a file or folder). */
export function commonFolder(paths: readonly string[], isFolder: (p: string) => boolean):
    string {
  const folders = paths.map((p) => (isFolder(p) ? p : path.dirname(p)));
  let common = folders[0] ?? "";
  for (const folder of folders.slice(1)) {
    while (!isInside(common, folder)) {
      const parent = path.dirname(common);
      if (parent === common) {
        break;
      }
      common = parent;
    }
  }
  return common;
}

/** Sorts relative paths as code2pdf walks a folder: depth first, each folder's entries by
    name. */
export function treeOrder(a: string, b: string): number {
  const pa = a.split("/");
  const pb = b.split("/");
  for (let i = 0; i < Math.min(pa.length, pb.length); i++) {
    if (pa[i] !== pb[i]) {
      return pa[i] < pb[i] ? -1 : 1;
    }
  }
  return pa.length - pb.length;
}

/** The items of LIST in order, without repeats. */
export function unique<T>(list: readonly T[]): T[] {
  return [...new Set(list)];
}

/** A safe file name: letters, digits, dots, dashes and underscores only. */
export function fileNamePart(text: string): string {
  return text.replace(/[^A-Za-z0-9._-]+/g, "-").replace(/^-+|-+$/g, "") || "code";
}

```

```ts
/** The file's name without its last extension, as code2pdf names a single file's PDF. */
export function stem(file: string): string {
  const base = path.basename(file);
  const dot = base.lastIndexOf(".");
  return dot > 0 ? base.slice(0, dot) : base;
}

/** The PDF name for part of FILE: `app-lines-10-40.pdf`. */
export function selectionPdfName(file: string, ranges: readonly LineRange[]): string {
  const first = ranges[0]?.start ?? 1;
  const last = ranges[ranges.length - 1]?.end ?? first;
  return `${fileNamePart(stem(file))}-lines-${first}-${last}.pdf`;
}

/** Expands a leading `~` to HOME. */
export function expandHome(p: string, home: string): string {
  return p === "~" ? home : p.replace(/^~(?=[/\\])/, home);
}
```

src/test/integration/

commands.test.ts

TypeScript, 230 lines, last changed 2026-10-11

```ts
import { strict as assert } from "node:assert";
import { execFileSync } from "node:child_process";
import * as fs from "node:fs";
import * as os from "node:os";
import * as path from "node:path";
import * as vscode from "vscode";

const project = () => (vscode.workspace.workspaceFolders ?? [])[0].uri.fsPath;
const uri = (rel: string) => vscode.Uri.file(path.join(project(), rel));

/** The PDF's text, page by page as pdftotext reads it. */
function text(pdf: string): string {
  return execFileSync("pdftotext", ["-layout", pdf, "-"], { encoding: "utf8" });
}

function printed(pdf: string | undefined): string {
  assert.ok(pdf, "the command returned a PDF path");
  assert.ok(fs.existsSync(pdf), `${pdf} exists`);
  assert.ok(path.dirname(pdf).startsWith(process.env.CODE2PDF_TEST_PDFS ?? "?"), "saved in the outputFolder setting");
  return text(pdf);
}

type Window = Record<string, unknown>;

/** Replaces vscode.window.NAME while BODY runs. */
async function stubbed<T>(name: string, stub: (...a: never[]) => unknown, body: () => Thenable<T>): Promise<T> {
  const window = vscode.window as unknown as Window;
  const original = window[name];
  window[name] = stub;
  try {
    return await body();
  } finally {
    window[name] = original;
  }
}

/** Messages shown while BODY runs. */
async function messages(body: () => Promise<unknown>): Promise<string[]> {
  const shown: string[] = [];
  const record = (m: string) => {
    shown.push(m);
    return Promise.resolve(undefined);
  };
  await stubbed("showErrorMessage", record, () => stubbed("showWarningMessage", record, () => stubbed("showInformationMessage", record, body)));
  return shown;
}

describe("code2pdf", () => {
  before(async () => {
    await vscode.extensions.getExtension("readbysearch.code2pdf")?.activate();
```

```
});

beforeEach(() => {
  assert.ok(vscode.workspace.getConfiguration("code2pdf").get<string>("outputFolder"),
    "the fixture's outputFolder is set, so no test opens a real save dialog");
});

it("registers its commands, and version 1's", async () => {
  const all = await vscode.commands.getCommands(true);
  for (const id of ["code2pdf.print", "code2pdf.chooseFiles", "code2pdf.printFile",
 "code2pdf.printSelection",
    "code2pdf.printWorkspace", "code2pdf.printChanges", "extension.printFileToPdf",
 "extension.printAll"]) {
    assert.ok(all.includes(id), id);
  }
});

it("prints a file", async () => {
  const pdf = await vscode.commands.executeCommand<string>("code2pdf.printFile", uri("src/
 app.py"));
  assert.equal(path.basename(pdf), "app.pdf");
  const t = printed(pdf);
  assert.match(t, /def greet\(name\):/);
  assert.match(t, /^src\/\s+app\.py$/m, "labelled with its folder in the project");
});

it("prints the selected lines, with their own line numbers", async () => {
  const editor = await vscode.window.showTextDocument(uri("src/app.py"));
  editor.selections = [new vscode.Selection(9, 0, 12, 0), new vscode.Selection(24, 3, 24,
 8)];
  const pdf = await vscode.commands.executeCommand<string>("code2pdf.printSelection");
  assert.equal(path.basename(pdf), "app-lines-10-25.pdf");
  const t = printed(pdf);
  assert.match(t, /^\s*10\s+value_10 = 10/m);
  assert.match(t, /^\s*12\s+value_12 = 12/m);
  assert.match(t, /^\s*25\s+value_25 = 25/m);
  assert.doesNotMatch(t, /value_13 = 13/, "line 13 wasn't selected (the selection ends at
 its start)");
  assert.doesNotMatch(t, /def greet/);
});

it("prints the workspace folder with code2pdf's selection", async () => {
  const t = printed(await
 vscode.commands.executeCommand<string>("code2pdf.printWorkspace"));
  for (const file of ["README.md", "src/app.py", "src/util.py", "tests/test_app.py"]) {
    assert.ok(t.includes(file), file);
  }
  assert.doesNotMatch(t, /json_marker/, "JSON isn't in the default selection");
  assert.doesNotMatch(t, /code2pdf\.outputFolder/, "hidden folders are skipped");
});

it("uses the include and exclude settings", async () => {
  const config = vscode.workspace.getConfiguration("code2pdf");
  await config.update("include", ["*.json"], vscode.ConfigurationTarget.Workspace);
  await config.update("exclude", ["tests"], vscode.ConfigurationTarget.Workspace);
  try {
    const t = printed(await vscode.commands.executeCommand<string>("code2pdf.print",
 vscode.Uri.file(project())));
    assert.match(t, /json_marker/);
```

```
      assert.doesNotMatch(t, /tested greeting/);
    } finally {
      await config.update("include", undefined, vscode.ConfigurationTarget.Workspace);
      await config.update("exclude", undefined, vscode.ConfigurationTarget.Workspace);
    }
  });

  it("prints several selected files and folders as one PDF", async () => {
    const pdf = await vscode.commands.executeCommand<string>("code2pdf.print",
   uri("README.md"), [uri("tests"), uri("README.md"), uri("src/util.py")]);
    assert.equal(path.basename(pdf), "shop-selected.pdf");
    const t = printed(pdf);
    assert.match(t, /A tiny shop for tests/);
    assert.match(t, /util helper/);
    assert.match(t, /tested greeting/);
    assert.doesNotMatch(t, /def greet/);
    assert.ok(t.indexOf("A tiny shop") < t.indexOf("util helper") && t.indexOf("util
   helper") < t.indexOf("tested greeting"),
      "sorted by path");
  });

  it("prints the files left checked in the list", async () => {
    let offered: string[] = [];
    const pdf = await stubbed("showQuickPick", (items: readonly vscode.QuickPickItem[]) => {
      offered = items.map((i) => i.label);
      return Promise.resolve(items.filter((i) => i.label !== "app.py"));
    }, () => vscode.commands.executeCommand<string>("code2pdf.chooseFiles", uri("src")));
    assert.deepEqual(offered, ["app.py", "util.py"], "paths relative to the folder");
    const t = printed(pdf);
    assert.match(t, /util helper/);
    assert.doesNotMatch(t, /def greet/);
  });

  it("prints the last commit's changes, compared with the files as they are now", async ()
   => {
    const pdf = await vscode.commands.executeCommand<string>("code2pdf.printChanges", {
   folder: project(), rev: "HEAD~1" });
    const t = printed(pdf);
    assert.match(t, /^\s*20\s+value_20 = 'changed in commit two'/m);
    assert.match(t, /util helper, edited but not committed/, "compared with the files as
   they are now");
    assert.doesNotMatch(t, /value_5 = 5/, "only the changed part of a long file");
  });

  it("offers the changes to print, and prints the one picked", async () => {
    let labels: string[] = [];
    const pdf = await stubbed("showQuickPick", (items: readonly vscode.QuickPickItem[]) => {
      labels = items.filter((i) => i.kind !== vscode.QuickPickItemKind.Separator).map((i)
   => i.label);
      return Promise.resolve(items.find((i) => i.label === "Uncommitted changes"));
    }, () => vscode.commands.executeCommand<string>("code2pdf.printChanges"));
    assert.deepEqual(labels.slice(0, 3), ["Uncommitted changes", "The last commit", "Change
   value 20"]);
    assert.equal(path.basename(pdf), "shop-uncommitted.pdf");
    const t = printed(pdf);
    assert.match(t, /edited but not committed/);
    assert.doesNotMatch(t, /changed in commit two/);
  });

```

```
155  it("version 1's command still prints the open file", async () => {
156    await vscode.window.showTextDocument(uri("src/util.py"));
157    assert.match(printed(await
      vscode.commands.executeCommand<string>("extension.printFileToPdf")), /util helper/);
158  });
159
160  it("shows code2pdf's error instead of failing silently", async () => {
161    const config = vscode.workspace.getConfiguration("code2pdf");
162    await config.update("theme", "no-such-theme", vscode.ConfigurationTarget.Workspace);
163    try {
164      let pdf: string | undefined = "not run";
165      const shown = await messages(async () => {
166        pdf = await vscode.commands.executeCommand<string>("code2pdf.printFile", uri("src/
      util.py"));
167      });
168      assert.equal(pdf, undefined);
169      assert.ok(shown.some((m) => m.includes("unknown theme 'no-such-theme'")),
      shown.join(" | "));
170    } finally {
171      await config.update("theme", undefined, vscode.ConfigurationTarget.Workspace);
172    }
173  });
174
175  it("says so when code2pdf isn't where the setting points", async () => {
176    const config = vscode.workspace.getConfiguration("code2pdf");
177    const outputFolder = config.inspect<string>("outputFolder")?.workspaceValue;
178    await config.update("executable", "/nonexistent/code2pdf",
      vscode.ConfigurationTarget.Global);
179    await config.update("outputFolder", "", vscode.ConfigurationTarget.Workspace);
180    try {
181      let asked = false;
182      const shown = await messages(() => stubbed("showSaveDialog", () => {
183        asked = true;
184        return Promise.resolve(undefined);
185      }, async () => {
186        assert.equal(await vscode.commands.executeCommand<string>("code2pdf.printFile",
      uri("src/util.py")), undefined);
187      }));
188      assert.ok(shown.some((m) => m.includes("isn't at /nonexistent/code2pdf")),
      shown.join(" | "));
189      assert.ok(!asked, "doesn't ask where to save a PDF it can't make");
190    } finally {
191      await config.update("executable", undefined, vscode.ConfigurationTarget.Global);
192      await config.update("outputFolder", outputFolder,
      vscode.ConfigurationTarget.Workspace);
193    }
194  });
195
196  it("keeps the elapsed time moving while code2pdf is quiet", async () => {
197    const fake = path.join(fs.mkdtempSync(path.join(os.tmpdir(), "code2pdf-fake-")),
      "code2pdf");
198    fs.writeFileSync(fake, [
199      "#!/bin/sh",
200      'if [ "$1" = "--version" ]; then echo "code2pdf 3.12 (commit x, 2026-10-11)"; echo "
      installed: Homebrew, /x"; exit 0; fi',
201      'out=""; prev=""; for a in "$@"; do [ "$prev" = "-o" ] && out="$a"; prev="$a"; done',
202      'echo "Converting to a PDF file for src/util.py"',
203      'mkdir -p "$(dirname "$out")"',
204      "sleep 2.5",
```

```
205       "printf '%%PDF-1.7\\n' > \"$out\"",
206       'echo "PDF created at $out"',
207     ].join("\n"), { mode: 0o755 });
208     const config = vscode.workspace.getConfiguration("code2pdf");
209     await config.update("executable", fake, vscode.ConfigurationTarget.Global);
210     const reports: string[] = [];
211     try {
212       const pdf = await stubbed("withProgress", (_options: unknown, task: (p: unknown, t:
    unknown) => Thenable<unknown>) =>
213         task({ report: (r: { message?: string }) => reports.push(r.message ?? "") },
214           { isCancellationRequested: false, onCancellationRequested: () => ({ dispose()
    {} }) }),
215         () => vscode.commands.executeCommand<string>("code2pdf.printFile", uri("src/
    util.py")));
216       assert.ok(pdf && fs.existsSync(pdf));
217     } finally {
218       await config.update("executable", undefined, vscode.ConfigurationTarget.Global);
219     }
220     assert.ok(reports.includes("src/util.py (0:00)"), reports.join(" | "));
221     assert.ok(reports.some((r) => /^src\/util\.py \(0:0[12]\)$/.test(r)), `ticks while
    quiet: ${reports.join(" | ")}`);
222   });
223
224   it("says what to do outside a git repository", async () => {
225     const shown = await messages(async () => {
226       assert.equal(await vscode.commands.executeCommand<string>("code2pdf.printChanges", {
    folder: path.dirname(project()) }), undefined);
227     });
228     assert.ok(shown.some((m) => m.includes("isn't in a git repository")), shown.join(" |
    "));
229   });
230 });
```

src/test/manual/

viewer.test.ts

TypeScript, 20 lines, last changed 2026-10-11

```ts
import { strict as assert } from "node:assert";
import * as path from "node:path";
import * as vscode from "vscode";

// Run by hand (it opens a PDF on your screen):
//   CODE2PDF_TEST_FILES='out/test/manual/*.test.js' npx vscode-test
describe("viewer (manual check, not in CI)", () => {
  it("opens the PDF in the system viewer", async () => {
    const config = vscode.workspace.getConfiguration("code2pdf");
    await config.update("openWith", "viewer", vscode.ConfigurationTarget.Workspace);
    try {
      const folder = (vscode.workspace.workspaceFolders ?? [])[0].uri.fsPath;
      const pdf = await vscode.commands.executeCommand<string>("code2pdf.printFile",
    vscode.Uri.file(path.join(folder, "src/util.py")));
      assert.ok(pdf);
      console.log(`Printed ${pdf}: check that it opened in the PDF viewer`);
    } finally {
      await config.update("openWith", undefined, vscode.ConfigurationTarget.Workspace);
    }
  });
});
```

src/test/unit/

args.test.ts

TypeScript, 60 lines, last changed 2026-10-11

```ts
import { strict as assert } from "node:assert";
import { execFileSync } from "node:child_process";
import { test } from "node:test";
import * as args from "../../args";
import { installed } from "./helpers";

const plain: args.PrintSettings = { theme: "paper", include: [], exclude: [], legacy: {} };

test("a file, with the default theme left out", () => {
  assert.deepEqual(args.fileArgs("src/app.py:10-20", "/o/app.pdf", plain), ["-s", "src/app.py:10-20", "-o", "/o/app.pdf"]);
  assert.deepEqual(args.fileArgs("app.py", "/o/a.pdf", { ...plain, theme: " mono " }), ["-s", "app.py", "-o", "/o/a.pdf", "--theme", "mono"]);
});

test("a folder, with include and exclude globs", () => {
  const settings = { ...plain, include: ["*.json", " "], exclude: ["tests", "*_test.go"] };
  assert.deepEqual(args.folderArgs(".", "/o/p.pdf", settings),
    ["-a", ".", "-o", "/o/p.pdf", "--include", "*.json", "--exclude", "tests", "--exclude", "*_test.go"]);
  assert.deepEqual(args.listArgs("src", settings), ["-a", "src", "--list-files", "--include", "*.json", "--exclude", "tests", "--exclude", "*_test.go"]);
});

test("a list of files takes no selection options (code2pdf refuses them)", () => {
  const settings = { ...plain, include: ["*.json"], exclude: ["tests"] };
  assert.deepEqual(args.filesFromArgs(".", "/o/p.pdf", settings), ["-a", ".", "--files-from", "-", "-o", "/o/p.pdf"]);
});

test("changes take exclude, but not include (code2pdf refuses it)", () => {
  const settings = { ...plain, include: ["*.json"], exclude: ["tests"], theme: "mono" };
  assert.deepEqual(args.changedArgs("HEAD~1", ".", "/o/c.pdf", settings),
    ["--changed", "HEAD~1", "-a", ".", "-o", "/o/c.pdf", "--theme", "mono", "--exclude", "tests"]);
});

test("version 1's settings, when set, keep their meaning", () => {
  assert.deepEqual(args.legacyArgs({}), []);
  assert.deepEqual(args.legacyArgs({
    blacklistedFolders: ["node_modules", "gen"],
    blacklistedFolderPattern: "tmp*",
    whitelistedFileExtensions: ["py", ".sh", "*.js"],
    whitelistedFileNames: ["README.md", "launch.json"],
  }), ["--ignore-folders", "node_modules,gen", "--exclude", "tmp*", "--include-types", "py,sh,js",
    "--include", "README.md", "--include", "launch.json"]);
  assert.deepEqual(args.legacyArgs({ blacklistedFolderPattern: "env*" }), [], "code2pdf skips env* folders anyway");
});

test("command lines in the log are quoted for a shell", () => {
```

```
  assert.equal(args.shellQuote(["code2pdf", "-a", ".", "--exclude", "*_test.go", "-o", "/tmp/my file.pdf", "--summary", "it's"]),
    "code2pdf -a . --exclude '*_test.go' -o '/tmp/my file.pdf' --summary 'it'\\''s'");
});

test("every option used here is in code2pdf --help", { skip: !installed() && "code2pdf isn't installed" }, () => {
  const help = execFileSync("code2pdf", ["--help"], { encoding: "utf8" });
  const settings = { theme: "mono", include: ["*.json"], exclude: ["tests"], legacy: {
    blacklistedFolders: ["x"], blacklistedFolderPattern: "y*", whitelistedFileExtensions: ["py"], whitelistedFileNames: ["z"] } };
  const used = new Set([
    ...args.fileArgs("a.py", "o.pdf", settings), ...args.folderArgs(".", "o.pdf", settings), ...args.listArgs(".", settings),
    ...args.filesFromArgs(".", "o.pdf", settings), ...args.changedArgs("HEAD", ".", "o.pdf", settings),
  ].filter((a) => a.startsWith("-") && a !== "-"));
  for (const option of used) {
    assert.match(help, new RegExp(`(^|[\\s,])${option}(?=[\\s,=]|$)`, "m"), `${option} isn't in code2pdf --help`);
  }
});
```

src/test/unit/

cli.test.ts

TypeScript, 106 lines, last changed 2026-10-11

```ts
import { strict as assert } from "node:assert";
import * as fs from "node:fs";
import * as path from "node:path";
import { test } from "node:test";
import * as cli from "../../cli";
import { script, tempDir } from "./helpers";

const BREW = `code2pdf 3.12 (commit 2e8b92f, 2026-10-11)
  installed: Homebrew, /opt/homebrew/Cellar/code2pdf/3.12
  python 3.14.8, weasyprint 70.0, pygments 2.21.0, pypdf 6.20.0
  CJK font: PingFang SC
`;

test("version output from each kind of install", () => {
  assert.deepEqual(cli.parseVersion(BREW), { version: "3.12", installKind: "Homebrew", installDir: "/opt/homebrew/Cellar/code2pdf/3.12" });
  assert.deepEqual(cli.parseVersion("code2pdf 3.11.1 (commit a, 2026-10-10)\n  installed: Debian package, /usr/share/code2pdf\n"), { version: "3.11.1", installKind: "Debian package", installDir: "/usr/share/code2pdf" });
  assert.deepEqual(cli.parseVersion("code2pdf 3.12+3 (commit b, 2026-10-12)\n  installed: install.sh (conda-forge environment), /home/u/.local/share/code2pdf\n"), { version: "3.12+3", installKind: "install.sh (conda-forge environment)", installDir: "/home/u/.local/share/code2pdf" });
  assert.equal(cli.parseVersion("code2pdf (version unknown: python3 not found), /usr/share/code2pdf")?.version, "unknown");
  assert.equal(cli.parseVersion("Usage: code2pdf [OPTIONS] -s FILE"), undefined);
});

test("3.12 or later is new enough; checkouts without a version count as new", () => {
  for (const v of ["3.12", "3.12.1", "3.12+3", "3.13", "4.0", "unreleased", "unknown"]) {
    assert.ok(cli.isNewEnough(v), v);
  }
  for (const v of ["3.11", "3.11.1", "3.9", "2.0", "3.1"]) {
    assert.ok(!cli.isNewEnough(v), v);
  }
});

test("where code2pdf is looked for", () => {
  assert.deepEqual(cli.candidates("~/bin/code2pdf", "/home/u", "linux"), ["/home/u/bin/code2pdf"]);
  const usual = cli.candidates("", "/home/u", "linux");
  assert.equal(usual[0], "code2pdf", "PATH first");
  assert.ok(usual.includes("/opt/homebrew/bin/code2pdf"));
  assert.ok(usual.includes(path.join("/home/u", ".local", "bin", "code2pdf")));
});

test("find: a working code2pdf, an old one, and none", async () => {
  const dir = tempDir();
  const good = script(dir, "good", `cat <<'EOF'\n${BREW}EOF`);
  assert.equal((await cli.find(good)).version, "3.12");

```

```
46    const old = script(dir, "old", `echo "code2pdf 3.11 (commit x, 2026-10-10)"; echo "
       installed: Homebrew, /x"`);
47    await assert.rejects(cli.find(old), (e: unknown) => e instanceof cli.TooOld &&
       e.found.version === "3.11");
48
49    // Before 3.7, --version wasn't an option: the usage text comes back instead
50    const ancient = script(dir, "ancient", `echo "Usage: code2pdf [OPTIONS] -s FILE"; exit
       1`);
51    await assert.rejects(cli.find(ancient), (e: unknown) => e instanceof cli.TooOld);
52
53    await assert.rejects(cli.find(path.join(dir, "missing")), (e: unknown) => e instanceof
       cli.NotInstalled && e.fromSetting);
54  });
55
56  test("run: stdin in, stdout and progress out, exit code", async () => {
57    const dir = tempDir();
58    const fake = script(dir, "fake", [
59      `echo "code2pdf: 2 files, 40 lines (~2 pages)" >&2`,
60      `echo "  [2/3] Laying out page 1 of ~2 (50%), 0:00 elapsed" >&2`,
61      `cat`,
62      `echo "Done: 2 pages in 0:00 → out.pdf" >&2`,
63      `exit 3`,
64    ].join("\n"));
65    const updates: (number | undefined)[] = [];
66    const result = await cli.run(fake, { cwd: dir, args: [], input: "a.py\nb.py\n",
       quietStdout: true, onProgress: (u) => updates.push(u.fraction) });
67    assert.equal(result.code, 3);
68    assert.equal(result.stdout, "a.py\nb.py\n");
69    assert.equal(result.reader.pages, 2);
70    assert.deepEqual(updates, [0, 0.05 + 0.85 * 0.5, 1]);
71    assert.equal(result.cancelled, false);
72  });
73
74  function cancelAfter(ms: number): cli.Cancellation {
75    const listeners: (() => void)[] = [];
76    const token = { isCancellationRequested: false, onCancellationRequested: (l: () => void)
       => listeners.push(l) };
77    setTimeout(() => {
78      token.isCancellationRequested = true;
79      listeners.forEach((l) => l());
80    }, ms);
81    return token;
82  }
83
84  test("cancel sends Ctrl-C to code2pdf and the processes it started", async () => {
85    const dir = tempDir();
86    const marker = path.join(dir, "child-stopped");
87    // Like bin/code2pdf: a shell script whose child does the work and cleans up on SIGINT
88    const child = script(dir, "child", `trap 'touch "${marker}"; echo "code2pdf: stopped
       (SIGINT); nothing was written" >&2; exit 1' INT\nwhile :; do sleep 0.1; done`);
89    const parent = script(dir, "parent", `"${child}"`);
90    const started = Date.now();
91    const result = await cli.run(parent, { cwd: dir, args: [], cancellation:
       cancelAfter(300), graceMs: 10_000 });
92    assert.equal(result.cancelled, true);
93    assert.ok(fs.existsSync(marker), "the child got SIGINT");
94    assert.ok(Date.now() - started < 5000, "stopped without waiting for the kill");
95  });
96
```

```
97  test("cancel kills code2pdf if Ctrl-C doesn't stop it", async () => {
98    const dir = tempDir();
99    const stubborn = script(dir, "stubborn", `trap '' INT\nwhile :; do sleep 0.1; done`);
100   const result = await cli.run(stubborn, { cwd: dir, args: [], cancellation:
       cancelAfter(200), graceMs: 300 });
101   assert.equal(result.cancelled, true);
102 });
103
104 test("run rejects when the command doesn't exist", async () => {
105   await assert.rejects(cli.run("/nonexistent/code2pdf", { cwd: "/", args: [] }), /ENOENT/);
106 });
```

src/test/unit/

git.test.ts

TypeScript, 99 lines, last changed 2026-10-11

```ts
import { strict as assert } from "node:assert";
import { execFileSync } from "node:child_process";
import * as fs from "node:fs";
import * as path from "node:path";
import { test } from "node:test";
import * as git from "../../git";
import { gitIn, installed, tempDir, write } from "./helpers";

function repo(): string {
  const dir = path.join(tempDir(), "shop");
  fs.mkdirSync(dir);
  gitIn(dir, "init", "-q", "-b", "main");
  return dir;
}

test("a repository without commits offers nothing", async () => {
  const dir = repo();
  write(dir, "a.py", "x = 1\n");
  assert.deepEqual(await git.changeChoices(dir), { main: [], commits: [] });
  assert.equal(await git.repositoryRoot(dir), fs.realpathSync(dir));
});

test("one commit: nothing to compare it with, since code2pdf can't diff a first commit",
    async () => {
  const dir = repo();
  write(dir, "a.py", "x = 1\n");
  gitIn(dir, "add", ".");
  gitIn(dir, "commit", "-qm", "first");
  assert.deepEqual(await git.changeChoices(dir), { main: [], commits: [] });
});

test("uncommitted changes, the last commit, and commits back to now", async () => {
  const dir = repo();
  write(dir, "a.py", "x = 1\n");
  gitIn(dir, "add", ".");
  gitIn(dir, "commit", "-qm", "first");
  const first = gitIn(dir, "rev-parse", "HEAD");
  write(dir, "a.py", "x = 2\n");
  gitIn(dir, "commit", "-qam", "Second: change x");
  const second = gitIn(dir, "rev-parse", "HEAD");
  write(dir, "a.py", "x = 3\n");
  write(dir, "new.py", "y = 1\n");

  const { main, commits } = await git.changeChoices(dir);
  assert.deepEqual(main.map((c) => c.label), ["Uncommitted changes", "The last commit"]);
  assert.equal(main[0].rev, "HEAD");
  assert.match(main[0].description ?? "", /1 file changed/);
  assert.match(main[0].detail ?? "", /\(1\)/, "the untracked file is mentioned");
  assert.equal(main[1].rev, first);
  assert.match(main[1].description ?? "", /Second: change x$/);
  assert.match(main[1].detail ?? "", /uncommitted/);
  assert.equal(main[1].name, `shop-${second.slice(0, 7)}`);
```

```
52    // Picking a commit prints it and what came after it, so the first commit (no parent)
       isn't offered
53    assert.deepEqual(commits.map((c) => [c.label, c.rev]), [["Second: change x", first]]);
54
55    assert.ok(await git.isCommit(dir, "HEAD~1"));
56    assert.ok(!(await git.isCommit(dir, "nope")));
57    assert.ok(!(await git.isCommit(dir, "--all")));
58  });
59
60  test("a branch: its changes since it left main", async () => {
61    const dir = repo();
62    write(dir, "a.py", "x = 1\n");
63    gitIn(dir, "add", ".");
64    gitIn(dir, "commit", "-qm", "first");
65    write(dir, "a.py", "x = 2\n");
66    gitIn(dir, "commit", "-qam", "second");
67    const fork = gitIn(dir, "rev-parse", "HEAD");
68    gitIn(dir, "checkout", "-qb", "feature/login");
69    write(dir, "b.py", "y = 1\n");
70    gitIn(dir, "add", ".");
71    gitIn(dir, "commit", "-qm", "login");
72    write(dir, "b.py", "y = 2\n");
73    gitIn(dir, "commit", "-qam", "login 2");
74
75    const { main } = await git.changeChoices(dir);
76    const branch = main.find((c) => c.label.startsWith("This branch"));
77    assert.equal(branch?.label, "This branch, feature/login");
78    assert.equal(branch?.rev, fork);
79    assert.equal(branch?.description, "2 commits since it left main");
80    assert.equal(branch?.name, "shop-feature-login");
81
82    gitIn(dir, "checkout", "-q", "main");
83    assert.ok(!(await git.changeChoices(dir)).main.some((c) => c.label.startsWith("This
       branch")), "not on main itself");
84  });
85
86  test("each choice works with code2pdf --changed", { skip: !installed() && "code2pdf isn't
      installed" }, async () => {
87    const dir = repo();
88    write(dir, "a.py", "def f():\n    return 1\n");
89    gitIn(dir, "add", ".");
90    gitIn(dir, "commit", "-qm", "first");
91    write(dir, "a.py", "def f():\n    return 2\n");
92    gitIn(dir, "commit", "-qam", "second");
93    write(dir, "a.py", "def f():\n    return 3\n");
94    const { main, commits } = await git.changeChoices(dir);
95    for (const choice of [...main, ...commits]) {
96      const list = execFileSync("code2pdf", ["--changed", choice.rev, "--list-files"], { cwd:
       dir, encoding: "utf8" });
97      assert.match(list, /^a\.py:.*\+2-2/m, choice.label);
98    }
99  });
```

src/test/unit/

helpers.ts

TypeScript, 35 lines, last changed 2026-10-11

```ts
import { execFileSync } from "node:child_process";
import * as fs from "node:fs";
import * as os from "node:os";
import * as path from "node:path";

/** Whether the real code2pdf is on PATH (the tests that use it are skipped otherwise). */
export function installed(): boolean {
  try {
    execFileSync("code2pdf", ["--version"], { stdio: "ignore" });
    return true;
  } catch {
    return false;
  }
}

export function tempDir(prefix = "code2pdf-vscode-"): string {
  return fs.mkdtempSync(path.join(fs.realpathSync(os.tmpdir()), prefix));
}

/** An executable shell script at DIR/NAME. */
export function script(dir: string, name: string, body: string): string {
  const file = path.join(dir, name);
  fs.writeFileSync(file, `#!/bin/sh\n${body}\n`, { mode: 0o755 });
  return file;
}

export function gitIn(dir: string, ...args: string[]): string {
  return execFileSync("git", ["-c", "user.name=Test", "-c", "user.email=test@example.com",
    "-c", "commit.gpgsign=false", ...args],
    { cwd: dir, encoding: "utf8" }).trim();
}

export function write(dir: string, file: string, text: string): void {
  fs.mkdirSync(path.dirname(path.join(dir, file)), { recursive: true });
  fs.writeFileSync(path.join(dir, file), text);
}
```

src/test/unit/

install.test.ts

TypeScript, 38 lines, last changed 2026-10-11

```ts
import { strict as assert } from "node:assert";
import { test } from "node:test";
import { debWorks, installCommand, parseOsRelease, updateCommand } from "../../install";

const UBUNTU_24 = `PRETTY_NAME="Ubuntu 24.04.1 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
ID=ubuntu
ID_LIKE=debian
`;

test("os-release", () => {
  assert.deepEqual(parseOsRelease(UBUNTU_24), { id: "ubuntu", versionId: "24.04" });
  assert.ok(debWorks(parseOsRelease(UBUNTU_24)));
  assert.ok(!debWorks({ id: "ubuntu", versionId: "22.04" }));
  assert.ok(debWorks({ id: "debian", versionId: "13" }));
  assert.ok(!debWorks({ id: "debian", versionId: "12" }));
  assert.ok(!debWorks({ id: "fedora", versionId: "41" }));
});

test("install: Homebrew where there is one, else the .deb, else install.sh without sudo", () => {
  assert.equal(installCommand("darwin", true)?.command, "brew install readbysearch/tap/code2pdf");
  assert.equal(installCommand("darwin", false), undefined, "the guide explains Homebrew");
  assert.equal(installCommand("linux", true, parseOsRelease(UBUNTU_24))?.label, "Install with Homebrew");
  assert.match(installCommand("linux", false, parseOsRelease(UBUNTU_24))?.command ?? "", /sudo apt install \/tmp\/code2pdf_all\.deb$/);
  assert.match(installCommand("linux", false, { id: "ubuntu", versionId: "22.04" })?.command ?? "", /install\.sh \| bash -s -- --user$/);
  assert.match(installCommand("linux", false)?.command ?? "", /--user$/);
  assert.equal(installCommand("win32", false), undefined);
});

test("update the way it was installed", () => {
  const found = (installKind: string, installDir = "") => ({ command: "code2pdf", version: "3.11", installKind, installDir });
  assert.equal(updateCommand(found("Homebrew"), "/home/u")?.command, "brew upgrade readbysearch/tap/code2pdf");
  assert.match(updateCommand(found("Debian package"), "/home/u")?.command ?? "", /apt install/);
  assert.match(updateCommand(found("install.sh", "/home/u/.local/share/code2pdf"), "/home/u")?.command ?? "", /--user$/);
  assert.match(updateCommand(found("install.sh (conda-forge environment)", "/usr/local/share/code2pdf"), "/home/u")?.command ?? "", /--system$/);
  assert.equal(updateCommand(found("git checkout"), "/home/u"), undefined);
});
```

src/test/unit/

progress.test.ts

TypeScript, 104 lines, last changed 2026-10-11

```ts
import { strict as assert } from "node:assert";
import { test } from "node:test";
import { OutputReader, ProgressUpdate, clock, parseProgress, withElapsed } from "../../progress";

// What code2pdf 3.12 wrote to stderr for `code2pdf -a scripts -o a.pdf --exclude nothing_here`
const RUN = `Warning: --exclude 'nothing_here' matched nothing
code2pdf: 11 files, 2,918 lines (~62 pages)
  [1/3] Preparing layout…  0:00 elapsed
  [2/3] Laying out page 7 of ~62 (10%), 0:02 elapsed
  [2/3] Laying out page 31 of ~62 (50%), 0:03 elapsed
  [2/3] Laying out page 56 of ~62 (90%), 0:04 elapsed
  [3/3] Writing PDF…  0:05 elapsed, ~0:02 left
Done: 61 pages in 0:07 → /tmp/a.pdf
`;

function read(text: string, chunk = text.length) {
  const updates: ProgressUpdate[] = [];
  const reader = new OutputReader((u) => updates.push(u));
  for (let i = 0; i < text.length; i += chunk) {
    reader.feed(text.slice(i, i + chunk));
  }
  reader.end();
  return { reader, updates };
}

test("a whole run: progress goes up to 1, with the page count and the warning", () => {
  const { reader, updates } = read(RUN);
  assert.equal(reader.pages, 61);
  assert.deepEqual(reader.warnings, ["--exclude 'nothing_here' matched nothing"]);
  assert.equal(updates[0].message, "11 files, ~62 pages");
  const fractions = updates.map((u) => u.fraction).filter((f) => f !== undefined) as number[];
  assert.deepEqual([...fractions].sort((a, b) => a - b), fractions, "never goes backwards");
  assert.equal(fractions[fractions.length - 1], 1);
  assert.deepEqual(updates.map((u) => u.message).filter(Boolean).slice(1),
    ["Preparing layout", "Laying out page 7 of ~62", "Laying out page 31 of ~62", "Laying out page 56 of ~62", "Writing PDF"]);
});

test("lines split across chunks read the same", () => {
  const whole = read(RUN);
  for (const size of [1, 7, 64]) {
    const split = read(RUN, size);
    assert.deepEqual(split.updates, whole.updates);
    assert.equal(split.reader.pages, 61);
  }
});

test("parts mode", () => {
  assert.deepEqual(parseProgress("  in 4 parts of ~200 pages, 3 at a time"), { message: "In 4 parts" });
```

```
  const rendering = parseProgress("  [2/3] Rendering page 80 of ~800 (10%), 0:05 elapsed (parts done: 0/4)");
  assert.equal(rendering?.message, "Rendering page 80 of ~800");
  assert.ok(Math.abs((rendering?.fraction ?? 0) - 0.135) < 1e-9);
  assert.equal(parseProgress("  [3/3] Joining 4 parts…  0:30 elapsed")?.message, "Joining 4 parts");
});

test("-s reports on stdout", () => {
  const reader = new OutputReader();
  reader.feed("Converting to a PDF file for src/app.py:10-20\nPDF created at /tmp/app.pdf\n", "stdout");
  reader.end();
  assert.equal(parseProgress("Converting to a PDF file for src/app.py:10-20")?.message, "src/app.py:10-20");
  assert.equal(parseProgress("PDF created at /tmp/app.pdf")?.fraction, 1);
  assert.equal(reader.errorMessage(), undefined);
});

test("stdout and stderr keep their own partial lines", () => {
  const reader = new OutputReader();
  reader.feed("Warning: one", "stderr");
  reader.feed("Converting to a PDF", "stdout");
  reader.feed(" two\n", "stderr");
  reader.end();
  assert.deepEqual(reader.warnings, ["one two"]);
});

test("errors, notes, and warnings repeated after Done: are listed once", () => {
  const { reader } = read([
    "Warning: app.py:10-12 stops inside a block that goes on to line 20; to print it whole, use app.py:10-20",

    "Note: --changed HEAD: 1 untracked file(s) not included (git add them, or list them with --files-from): new.py",
    "Done: 2 pages in 0:01 → /tmp/x.pdf",
    "Warning: app.py:10-12 stops inside a block that goes on to line 20; to print it whole, use app.py:10-20",
  ].join("\n"));
  assert.equal(reader.warnings.length, 1);
  assert.equal(reader.notes.length, 1);
  assert.equal(reader.pages, 2);

  const failed = read("Error: unknown theme 'nosuch'; run `code2pdf --list-themes` to see the names\n").reader;
  assert.equal(failed.errorMessage(), "unknown theme 'nosuch'; run `code2pdf --list-themes` to see the names");
  assert.equal(read("File not found.\n").reader.errorMessage(), "File not found.");
  assert.equal(read("code2pdf: stopped (SIGINT); nothing was written\n").reader.errorMessage(),
    "code2pdf: stopped (SIGINT); nothing was written");
});

test("big numbers have commas", () => {
  const { reader, updates } = read("code2pdf: 1,204 files, 310,532 lines (~7,702 pages)\nDone: 7,689 pages in 6:01 → /tmp/big.pdf\n");
  assert.equal(updates[0].message, "1,204 files, ~7,702 pages");
  assert.equal(reader.pages, 7689);
```

```
95    assert.equal(parseProgress("code2pdf: 1 file, 11 lines (~2 pages)")?.message, "1 file, ~2
        pages");
96  });
97
98  test("elapsed time in the message, so a quiet code2pdf doesn't look frozen", () => {
99    assert.equal(clock(0), "0:00");
100   assert.equal(clock(75.9), "1:15");
101   assert.equal(clock(3600), "60:00");
102   assert.equal(withElapsed("Laying out page 7 of ~62", 21.4), "Laying out page 7 of ~62
        (0:21)");
103   assert.equal(withElapsed(undefined, 5), "0:05 elapsed");
104 });
```

src/test/unit/

specs.test.ts

TypeScript, 85 lines, last changed 2026-10-11

```ts
import { strict as assert } from "node:assert";
import { execFileSync } from "node:child_process";
import * as path from "node:path";
import { test } from "node:test";
import * as specs from "../../specs";
import { installed, tempDir, write } from "./helpers";

const sel = (startLine: number, endLine: number, endCharacter: number, isEmpty = false) =>
    ({ startLine, endLine, endCharacter, isEmpty });

test("a selection inside one line prints that line", () => {
  assert.deepEqual(specs.selectionRanges([sel(4, 4, 12)]), [{ start: 5, end: 5 }]);
});

test("whole lines selected by dragging end at the start of the next line, which isn't
    printed", () => {
  assert.deepEqual(specs.selectionRanges([sel(9, 12, 0)]), [{ start: 10, end: 12 }]);
  assert.deepEqual(specs.selectionRanges([sel(9, 12, 3)]), [{ start: 10, end: 13 }]);
});

test("several selections are sorted and merged; empty ones are ignored", () => {
  assert.deepEqual(specs.selectionRanges([sel(30, 34, 5), sel(2, 3, 1), sel(33, 40, 1),
    sel(4, 5, 2), sel(60, 60, 0, true)]),
     [{ start: 3, end: 6 }, { start: 31, end: 41 }]);
  assert.deepEqual(specs.selectionRanges([sel(7, 7, 0, true)]), []);
});

test("range specs as code2pdf reads them", () => {
  assert.equal(specs.rangeSpec("src/app.py", [{ start: 5, end: 5 }]), "src/app.py:5");
  assert.equal(specs.rangeSpec("src/app.py", [{ start: 1, end: 3 }, { start: 7, end: 9 }]),
    "src/app.py:1-3,7-9");
});

test("paths given to code2pdf are never taken for options", () => {
  assert.equal(specs.argPath("-weird.py"), "./-weird.py");
  assert.equal(specs.argPath("src/app.py"), "src/app.py");
  assert.equal(specs.argPath("."), ".");
});

test("relative paths and containment", () => {
  assert.equal(specs.relativePath("/p", "/p"), ".");
  assert.equal(specs.relativePath("/p", "/p/src/a.py"), "src/a.py");
  assert.ok(specs.isInside("/p", "/p/src"));
  assert.ok(specs.isInside("/p", "/p"));
  assert.ok(!specs.isInside("/p", "/pq/src"));
  assert.ok(!specs.isInside("/p/src", "/p"));
});

test("the folder that holds every selected item", () => {
  const folders = new Set(["/p", "/p/src", "/p/src/lib", "/p/docs"]);
  const isFolder = (p: string) => folders.has(p);
  assert.equal(specs.commonFolder(["/p/src/a.py", "/p/src/lib/b.py"], isFolder), "/p/src");
  assert.equal(specs.commonFolder(["/p/src/lib", "/p/docs/x.md"], isFolder), "/p");
```

```
  assert.equal(specs.commonFolder(["/p/src/lib"], isFolder), "/p/src/lib");
  assert.equal(specs.commonFolder(["/p/src/a.py"], isFolder), "/p/src");
});

test("tree order: depth first, by name, files and folders mixed", () => {
  const sorted = ["src/b.py", "README.md", "src/a/x.py", "a.py", "src/c.py", "bin/run",
    "CLAUDE.md", "a/z.py"].sort(specs.treeOrder);
  assert.deepEqual(sorted, ["CLAUDE.md", "README.md", "a/z.py", "a.py", "bin/run", "src/a/
    x.py", "src/b.py", "src/c.py"]);
  assert.deepEqual(["src/a.py", "src"].sort(specs.treeOrder), ["src", "src/a.py"]);
});

test("file names", () => {
  assert.equal(specs.stem("/x/app.test.py"), "app.test");
  assert.equal(specs.stem("/x/Makefile"), "Makefile");
  assert.equal(specs.stem("/x/.bashrc"), ".bashrc");
  assert.equal(specs.fileNamePart("feature/login fix"), "feature-login-fix");
  assert.equal(specs.fileNamePart("///"), "code");
  assert.equal(specs.selectionPdfName("/x/app.py", [{ start: 10, end: 40 }, { start: 90,
    end: 99 }]), "app-lines-10-99.pdf");
  assert.equal(specs.unique(["a", "b", "a"]).join(), "a,b");
});

test("~ means the home folder", () => {
  assert.equal(specs.expandHome("~/out", "/home/u"), path.join("/home/u", "out").replace(/\
    \/g, "/"));
  assert.equal(specs.expandHome("~", "/home/u"), "/home/u");
  assert.equal(specs.expandHome("/abs/~x", "/home/u"), "/abs/~x");
  assert.equal(specs.expandHome("~other/x", "/home/u"), "~other/x");
});

test("tree order matches code2pdf's own order", { skip: !installed() && "code2pdf isn't
   installed" }, () => {
  const dir = tempDir();
  for (const file of ["README.md", "Makefile", "a.py", "a/b.py", "src/b.py", "src/C.py",
    "src/a/x.py", "src/a/y/z.py", "tests/t.py", "_x.py"]) {
    write(dir, file, "x = 1\n");
  }
  const listed = execFileSync("code2pdf", ["-a", ".", "--list-files"], { cwd: dir,
    encoding: "utf8" }).trim().split("\n");
  assert.equal(listed.length, 10);
  assert.deepEqual([...listed].reverse().sort(specs.treeOrder), listed);
});
```